

```
==== src/redmail/__main__.py (125 строк) ====
1  from __future__ import annotations

3  import subprocess
4  import sys
5  from importlib.metadata import PackageNotFoundError, version

7  from PySide6.QtCore import Qt
8  from PySide6.QtGui import QColor, QFont, QPainter, QPixmap
9  from PySide6.QtWidgets import QApplication, QSplashScreen

11 from redmail import config_store
12 from redmail.ui import theme

14 _SPLASH_SIZE = (420, 240)
15 _SPLASH_BG = "#1a73e8"
16 _SPLASH_FG = "white"

19 def app_version() -> str:
20     """Версия для заставки и "О программе" – жалоба: "добавь в справку о
21     программе версию, сейчас там 0.0.1", т.е. одна и та же версия пакета
22     для КАЖДОЙ сборки (в pyproject.toml она не меняется между RPM-релизами,
23     там всегда "0.0.1" – реальный номер сборки живёт только в Release: у
24     .spec). На RED OS сначала спрашиваем сам установленный RPM-пакет
25     (даёт "0.0.1-30", ровно то, что нужно, чтобы отличить сборки друг от
26     друга); вне RED OS (разработка, другой дистрибутив) rpm просто нет –
27     тогда версия пакета Python как раньше."""
28     try:
29         result = subprocess.run(
30             ["rpm", "-q", "--queryformat", "%{VERSION}-%{RELEASE}", "redmail"],
31             capture_output=True,
32             text=True,
33             timeout=2,
34         )
35         if result.returncode == 0 and result.stdout.strip():
36             return result.stdout.strip()
37     except (OSError, subprocess.SubprocessError):
38         pass
39     try:
40         return version("redmail")
41     except PackageNotFoundError:
42         return "?"

45 def build_splash_pixmap(version: str) -> QPixmap:
46     """Заставка при запуске (жалоба: "открывай сразу информационное окно
47     до открытия основного окна... название, автора и ход загрузки") –
48     название/версия/автор совпадают с тем, что уже показывает "О
49     программе...", чтобы не заводить два разных источника правды."""
50     width, height = _SPLASH_SIZE
51     pixmap = QPixmap(width, height)
52     pixmap.fill(QColor(_SPLASH_BG))
53     painter = QPainter(pixmap)
54     painter.setRenderHint(QPainter.RenderHint.Antialiasing)
55     painter.setPen(QColor(_SPLASH_FG))

57     title_font = QFont()
58     title_font.setPointSize(22)
59     title_font.setBold(True)
60     painter.setFont(title_font)
61     painter.drawText(pixmap.rect().adjusted(24, 28, -24, 0), Qt.AlignmentFlag.AlignLeft |
Qt.AlignmentFlag.AlignTop, "RedMail")

63     text_font = QFont()
64     text_font.setPointSize(11)
65     painter.setFont(text_font)
66     lines = (
67         "Почтовый клиент для RED OS",
68         f"Версия {version}",
69         "Автор: Пономарев Роман Сергеевич",
70     )
71     for i, line in enumerate(lines):
72         painter.drawText(
73             pixmap.rect().adjusted(24, 74 + i * 22, -24, 0),
74             Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.AlignTop,
75             line,
76         )
77     painter.end()
78     return pixmap

81 def main() -> int:
82     app = QApplication(sys.argv)

84     # Жалоба (и после первой правки с repaint()): "информационное окно
```

```

85     # выводится не сразу, долго висит, потом появляется, и практически
86     # мгновенно открывается окно приложения" — то есть задержка была ДО
87     # показа заставки, а не после. Две причины: (1) `from
88     # redmail.ui.main_window import MainWindow` стоял на уровне модуля и
89     # тянул за собой PySide6.QtWebEngine* — инициализация Chromium занимает
90     # секунды ещё до входа в main(); теперь импорт отложен и сам является
91     # шагом "Загрузка интерфейса..." уже при видимой заставке; (2)
92     # app_version() запускает `rpm -q` (до 2 с на медленной VM) — тоже до
93     # показа. Заставка показывается сразу с версией "...", версия
94     # дорисовывается следом. repaint() — синхронная перерисовка: одного
95     # processEvents() недостаточно, чтобы окно гарантированно оказалось
96     # на экране до долгой блокировки потока (особенно X11 без композитора).
97     splash = QSplashScreen(build_splash_pixmap(""))
98     splash.show()
99     splash.repaint()
100    app.processEvents()

102    def report(message: str) -> None:
103        splash.showMessage(
104            message, Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.AlignBottom, QColor(_SPLASH_FG)
105        )
106        splash.repaint()
107        app.processEvents()

109    splash.setPixmap(build_splash_pixmap(app_version()))
110    report("Применение темы оформления...")
111    theme.apply_theme(app, config_store.load_theme())

113    report("Загрузка интерфейса...")
114    from redmail.ui.main_window import MainWindow

116    window = MainWindow()

118    report("Готово")
119    window.show()
120    splash.finish(window)
121    return app.exec()

124 if __name__ == "__main__":
125     raise SystemExit(main())

==== src/redmail/imap_client.py (833 строк) ====
1  from __future__ import annotations

3  import functools
4  import imaplib
5  import threading
6  from dataclasses import dataclass, field
7  from email import message_from_bytes
8  from email.header import decode_header
9  from email.message import Message

11 from imapclient import IMAPClient
12 from imapclient.exceptions import IMAPClientError

14 _HEADER_FIELDS = "BODY.PEEK[HEADER.FIELDS (IMPORTANCE X-PRIORITY)]"

16 # Флаг \Flagged ставим всегда вместе с цветом — так другие IMAP-клиенты
17 # (Thunderbird, сам Outlook по IMAP) увидят письмо помеченным, даже если не
18 # понимают наш собственный keyword с цветом. $-префикс — общепринятое
19 # соглашение для нестандартных keyword-флагов (как $Forwarded, $MDNSent).
20 MARKER_COLORS: dict[str, bytes] = {
21     "red": b"$RedMailRed",
22     "orange": b"$RedMailOrange",
23     "yellow": b"$RedMailYellow",
24     "green": b"$RedMailGreen",
25     "blue": b"$RedMailBlue",
26     "purple": b"$RedMailPurple",
27 }
28 _COLOR_BY_KEYWORD = {v: k for k, v in MARKER_COLORS.items()}

31 def split_markers(value: str | None) -> list[str]:
32     """Несколько маркеров на письмо (пожелание: "на письмо можно поставить
33     несколько маркеров") хранятся в том же поле marker_color через запятую
34     ("red,blue") — так не меняются схема кэша/архива и все места, где это
35     поле просто прокидывается дальше; разбирают его только те, кому нужны
36     отдельные цвета (иконка, фильтр, меню, IMAP-флаги)."""
37     return [color for color in (value or "").split(",") if color]

40 def join_markers(colors) -> str | None:
41     ordered: list[str] = []
42     for color in colors:
43         if color and color not in ordered:
44             ordered.append(color)
45     return ",".join(ordered) or None

```

```

47 # Таймаут одной операции на сожете IMAP (секунды). Не ограничивает
48 # длительность загрузки большого письма целиком — только паузу между
49 # порциями данных от сервера.
50 SOCKET_TIMEOUT = 120

52 # Сентинел по умолчанию для set_marker(previous_color=...) — отличает "вызывающий
53 # код не знает текущий маркер" (безопасный медленный путь: снять все
54 # возможные keyword'ы) от "previous_color=None" (точно знает, что маркера не
55 # было — снимать нечего). Спутать их означало бы на реальном сервере либо
56 # лишние round trip'ы, либо оставленный висеть старый keyword.
57 UNKNOWN_MARKER = object()

60 def _is_recoverable_by_reconnect(exc: Exception) -> bool:
61     """Протокольные ошибки (imaplib.IMAP4.error = IMAPClientError: команда
62     сервером понята, но отвергнута), которые всё же лечатся
63     переподключением — в отличие от остальных, см. _reconnecting:

65     * "command SELECT/... illegal in state NONAUTH" — сервер молча
66     разлогинил сессию после долгого простоя (жалоба: "после долгого
67     простоя выдаёт... лечится перезапуском"), но САМ TCP-сокеты при этом
68     мог и не порваться, поэтому это не OSError/EOFError/IMAP4.abort;
69     * "[UNAVAILABLE] Failed to open mailbox" / "Service temporarily
70     unavailable" (RFC 5530: временный отказ подсистемы сервера — жалоба:
71     "после сбоя сервера или принудительного простоя не восстанавливается
72     подключение, обновление даёт ошибку") — сессия после такого сбоя
73     сервера обычно уже невалидна, свежее соединение проходит.""
74     text = str(exc)
75     if "illegal in state" in text and "NONAUTH" in text:
76         return True
77     return "[UNAVAILABLE]" in text.upper()

80 def _reconnecting(method):
81     """После простоя реальный IMAP-сервер молча рвёт TCP-соединение (никто
82     не обязан держать сессию вечно — RFC 3501 не гарантирует этого), и
83     следующая же операция падала с сырой сетевой ошибкой (жалоба
84     пользователя: "после простоя часто выдаёт ошибку подключения").
85     OSError — общий предок и для обрыва соединения, и для TLS-ошибок в
86     современном Python.

88     imaplib.IMAP4.abort — ОТДЕЛЬНО от OSError, хотя семантически это тот
89     же случай: сам imaplib документирует его как "Service errors - close
90     and retry" (imaplib.py), и на практике так оборачивает разрыв TLS
91     ("EOF occurred in violation of protocol") при чтении строки ответа —
92     `class error(Exception)` в стандартной библиотеке НЕ наследуется от
93     OSError, так что раньше это вообще не попадало под переподключение
94     (жалоба: "периодически выдаёт ошибки" при обновлении/чтении письма —
95     ошибка показывалась как есть с первого же раза, без единой попытки
96     восстановить соединение). Настоящие протокольные ошибки
97     (IMAPClientError на команду, которую сервер понял, но отверг) НЕ
98     перехватываются — переподключение их не лечит, показываем как есть,
99     КРОМЕ "illegal in state NONAUTH" — см. _is_stale_session_error.""

101     @functools.wraps(method)
102     def wrapper(self, *args, **kwargs):
103         # Одно соединение — одна команда за раз. Тело письма грузится в
104         # фоновом потоке, а отметка "прочитано"/маркер/следующий fetch
105         # могли уйти в тот же сокет из другого потока параллельно; imaplib
106         # к этому не готов — ответы перемешиваются, и один из потоков ждёт
107         # своего ответа вечно (жалоба: "подвисает при переходе от письма к
108         # письму на загрузке письма"). RLock — потому что декорированные
109         # методы вызывают друг друга.
110         with self._lock:
111             try:
112                 return method(self, *args, **kwargs)
113             except (OSError, EOFError, imaplib.IMAP4.abort) as exc:
114                 try:
115                     self._reconnect()
116                 except Exception:
117                     raise exc from None # переподключиться тоже не вышло — исходная ошибка нагляднее
118                 return method(self, *args, **kwargs)
119             except imaplib.IMAP4.error as exc:
120                 if not _is_recoverable_by_reconnect(exc):
121                     raise
122                 try:
123                     self._reconnect()
124                 except Exception:
125                     raise exc from None
126                 return method(self, *args, **kwargs)

128     return wrapper

131 @dataclass
132 class Account:
133     host: str
134     username: str
135     password: str

```

```

136     port: int = 993
137     use_ssl: bool = True
138     # "password" — обычный LOGIN; "kerberos" — SSO для почтового сервера в
139     # домене: аутентификация идёт по Kerberos-билету, который ОС уже
140     # выдала при входе пользователя в домен (RED OS + SSSD), пароль в
141     # приложении не хранится и не используется (см. gssapi_sasl.py).
142     auth_type: str = "password"
143     # Необязательный keytab как источник билета для SSO (вместо билета из
144     # системного кэша) и principal, для которого он выписан — см.
145     # gssapi_sasl.acquire_credentials.
146     keytab_path: str = ""
147     principal: str = ""

150 @dataclass
151 class FolderInfo:
152     name: str
153     delimiter: str

156 @dataclass
157 class MessageSummary:
158     uid: int
159     subject: str
160     sender: str
161     sender_email: str
162     date: str
163     message_id: str
164     has_attachments: bool = False
165     marker_color: str | None = None
166     importance: str = "normal" # "high" | "normal" | "low"
167     is_read: bool = False
168     # Только для списка "Отправленные" — там "От кого" всегда сам
169     # пользователь, бесполезная колонка (жалоба: "в отправленных нет поля
170     # адресат, невозможно понять кому писали"). Достаётся бесплатно — те же
171     # данные ENVELOPE, что уже фетчатся для sender, просто ещё одно поле.
172     to: str = ""
173     # Стандартный IMAP-флаг \Answered — раньше нигде не читался и не
174     # проставлялся (жалоба: "если мы ответили на письмо, это никак не
175     # отражается, нужен какой-то признак").
176     is_answered: bool = False

179 @dataclass
180 class Attachment:
181     filename: str
182     content_type: str
183     payload: bytes

185     @property
186     def size(self) -> int:
187         return len(self.payload)

190 @dataclass
191 class MessageContent:
192     text: str
193     attachments: list[Attachment] = field(default_factory=list)
194     html: str = ""
195     # Content-Id (без угловых скобок) -> (content_type, данные) — картинки,
196     # встроенные в HTML через , а не обычные вложения.
197     inline_images: dict[str, tuple[str, bytes]] = field(default_factory=dict)
198     # Реквизиты письма (тема/отправитель/получатели) — раньше нигде не
199     # показывались при просмотре письма (жалоба: "невидно его реквизитов
200     # (тема, отправитель, адресаты)"). MessageSummary уже даёт subject/
201     # sender для списка писем, но не даёт To/Сс — они здесь.
202     subject: str = ""
203     from_: str = ""
204     to: str = ""
205     cc: str = ""
206     bcc: str = ""

209 class ImapSession:
210     """Одно живое IMAP-соединение на всё время работы с ящиком.

212     Открывать новое соединение (TCP + TLS + логин) на каждый клик по папке
213     или письму — секунды задержки на медленной сети. Здесь соединение
214     держится, пока пользователь не переподключится или не закроет окно.
215     """

217     def __init__(self, account: Account):
218         self.account = account
219         self.lock = threading.RLock()
220         self._client = self._new_client()
221         self._login()
222         self._selected_folder: str | None = None
223         self._selected_exists = 0
224         self._raw_folders: list[tuple] = []

```

```

226     def _login(self) -> None:
227         if self.account.auth_type == "kerberos":
228             # Импорт внутри функции: пакет gssapi требует системных
229             # библиотек Kerberos, которых нет на части машин (Windows,
230             # окружения без домена) – обычный пароль не должен ломаться
231             # из-за отсутствия зависимости, нужной только для SSO.
232             from redmail import gssapi_sasl

233             gssapi_sasl.imap_sasl_login(
234                 self._client,
235                 self.account.host,
236                 self.account.username,
237                 keytab_path=self.account.keytab_path,
238                 principal=self.account.principal,
239             )
240         else:
241             self._client.login(self.account.username, self.account.password)

242     def _new_client(self) -> IMAPClient:
243         # Таймаут на сожете обязателен: без него любое зависшее чтение
244         # (сервер молча перестал отвечать, рассинхрон ответов) блокирует
245         # поток навсегда – с таймаутом это OSError, которую _reconnecting
246         # лечит переподключением и повтором.
247         return IMAPClient(self.account.host, port=self.account.port, ssl=self.account.use_ssl,
248             timeout=SOCKET_TIMEOUT)

249     def close(self) -> None:
250         with self._lock:
251             try:
252                 self._client.logout()
253             except Exception:
254                 pass

255     def _reconnect(self) -> None:
256         self._client = self._new_client()
257         self._login()
258         if self._selected_folder is not None:
259             # Кое-что из вызывающего кода (fetch_summaries) не делает
260             # собственный SELECT – полагается, что папка уже выбрана
261             # предыдущим folder_message_count()/_select(). Восстанавливаем
262             # это состояние сразу, иначе повтор упал бы снова, уже по
263             # другой причине (ничего не выбрано).
264             self._client.select_folder(self._selected_folder, readonly=False)

265     def __enter__(self) -> "ImapSession":
266         return self

267     def __exit__(self, *exc_info) -> None:
268         self.close()

269     @reconnecting
270     def list_folders(self) -> list[FolderInfo]:
271         # Сырой ответ запоминаем – из него же достаём папку "Корзина" в
272         # trash_folder(), без второго похода на сервер (find_special_folder
273         # библиотеки сам заново вызывает list_folders).
274         self._raw_folders = self._client.list_folders()
275         return [
276             FolderInfo(name=name, delimiter=(delimiter or b"/").decode("ascii", errors="replace"))
277             for flags, delimiter, name in self._raw_folders
278             if b"\Noselect" not in flags
279         ]

280     @reconnecting
281     def create_folder(self, name: str) -> None:
282         self._client.create_folder(name)

283     @reconnecting
284     def rename_folder(self, old_name: str, new_name: str) -> None:
285         self._client.rename_folder(old_name, new_name)

286     def trash_folder(self) -> str | None:
287         return self._special_folder(b"\Trash", ("trash", "корзин"))

288     def sent_folder(self) -> str | None:
289         return self._special_folder(b"\Sent", ("sent", "отправленн"))

290     def drafts_folder(self) -> str | None:
291         return self._special_folder(b"\Drafts", ("draft", "черновик"))

292     def _special_folder(self, special_use_flag: bytes, name_hints: tuple[str, ...]) -> str | None:
293         # Сначала – SPECIAL-USE (RFC 6154), это надёжно, сервер сам сказал,
294         # какая папка какая. Но объявляет его не каждый реальный сервер –
295         # без запасного варианта по имени такая папка оставалась вовсе не
296         # распознанной: письмо можно было открыть, но не отправить (жалоба
297         # "из черновика не даёт отправить" – двойной клик по письму в
298         # "Черновиках" тихо считал, что это просто обычная папка, и
299         # открывал письмо на просмотр, а не на редактирование).
300         for flags, _delimiter, name in self._raw_folders:

```

```

313         if special_use_flag in flags:
314             return name
315     for _flags, _delimiter, name in self._raw_folders:
316         lowered = name.lower()
317         if any(hint in lowered for hint in name_hints):
318             return name
319     return None

321 @reconnecting
322 def append_message(self, folder: str, raw: bytes, *, flags: tuple[bytes, ...] = ()) -> None:
323     """Кладёт готовое (уже собранное) сообщение в папку напрямую,
324     минуя SMTP – для "Отправленные" (сервер сам не всегда сохраняет
325     копию исходящих) и "Черновики" (письмо, которое никуда не
326     отправлялось)."""
327     self._client.append(folder, raw, flags=flags)

329 @reconnecting
330 def folder_message_count(self, folder: str) -> int:
331     """SELECT папку, вернуть общее число писем в ней (EXISTS).

332     Дешёвая операция (без сканирования, в отличие от SEARCH) – на ней
333     удобно проверять, изменилось ли что-то в папке с прошлого раза,
334     прежде чем платить за полный FETCH сводок (см. CachedMailbox).
335     """
336     status = self._client.select_folder(folder, readonly=False)
337     self._selected_folder = folder
338     self._selected_exists = status[b"EXISTS"]
339     return self._selected_exists

342 @reconnecting
343 def folder_unseen_count(self, folder: str) -> int:
344     """Число непрочитанных писем в папке – жалоба: "подписывать
345     количество писем" рядом с папкой в дереве. STATUS, а не
346     select_folder/SEARCH: не переключает "текущую" папку сессии
347     (folder_message_count это делает) и не сканирует письма – можно
348     дёшево опросить сразу много папок подряд при построении дерева."""
349     status = self._client.folder_status(folder, ["UNSEEN"])
350     for key, value in status.items():
351         key_name = key.decode("ascii", errors="replace") if isinstance(key, bytes) else str(key)
352         if key_name.upper() == "UNSEEN":
353             return int(value)
354     return 0

356 @reconnecting
357 def fetch_summaries(self, limit: int = 50) -> list[MessageSummary]:
358     """Сводки последних `limit` писем уже выбранной папки.

359     Требуется, чтобы перед этим была вызвана folder_message_count –
360     отдельного SELECT здесь больше нет.
361     """
362     total = self._selected_exists
363     if total == 0:
364         return []
365     start = max(1, total - limit + 1)

366     # Порядковые номера, а не UID – иначе пришлось бы всё равно узнавать
367     # реальные UID через SEARCH. UID запрашиваем отдельным полем: он
368     # возвращается независимо от режима нумерации.
369     self._client.use_uid = False
370     try:
371         response = self._client.fetch(
372             f"{start}:*", ["ENVELOPE", "UID", "FLAGS", "BODYSTRUCTURE", _HEADER_FIELDS]
373         )
374     finally:
375         self._client.use_uid = True

376     by_seq = sorted(response.items(), key=lambda item: item[0], reverse=True)
377     return [_to_summary(data) for _seq, data in by_seq]

382 def fetch_folder_summaries(self, folder: str = "INBOX", limit: int = 50) -> list[MessageSummary]:
383     self.folder_message_count(folder)
384     return self.fetch_summaries(limit)

386 @reconnecting
387 def search_uids(self, folder: str, *, before=None) -> list[int]:
388     """UID всех писем папки (или только тех, что старше даты `before`) –
389     в отличие от fetch_summaries/fetch_folder_summaries это не
390     ограничено последними `limit` письмами: нужно для массовой
391     выгрузки в архив (вся папка / всё до даты), где важна ПОЛНАЯ папка,
392     а не то, что сейчас показано в таблице."""
393     self._select(folder)
394     criteria = ["BEFORE", before.strftime("%d-%b-%Y")] if before else "ALL"
395     return list(self._client.search(criteria))

397 def fetch_message_content(self, folder: str, uid: int) -> MessageContent:
398     return extract_content(message_from_bytes(self.fetch_message_raw(folder, uid)))

400 @reconnecting
401 def fetch_message_raw(self, folder: str, uid: int) -> bytes:

```

```

402     """Полный RFC 822 письма как есть — нужен для выгрузки в архив без
403     потерь (в отличие от fetch_message_content, который уже разобрал бы
404     текст/вложения и потерял бы всё остальное, например точные заголовки)."""
405     self._select(folder)
406     response = self._client.fetch([uid], [b"BODY.PEEK[]"])
407     return response[uid][b"BODY[]"]

409 @_reconnecting
410 def set_read(self, folder: str, uid: int, read: bool) -> None:
411     self._select(folder)
412     if read:
413         self._client.add_flags([uid], [b"\\Seen"])
414     else:
415         self._client.remove_flags([uid], [b"\\Seen"])

417 @_reconnecting
418 def set_answered(self, folder: str, uid: int) -> None:
419     """Ставит стандартный флаг \\Answered на письмо, на которое только
420     что был отправлен ответ — не снимается обратно (как и в других
421     почтовых клиентах, "ответили" необратимо для этого письма)."""
422     self._select(folder)
423     self._client.add_flags([uid], [b"\\Answered"])

425 @_reconnecting
426 def set_marker(self, folder: str, uid: int, color: str | None, *, previous_color=UNKNOWN_MARKER) -> None:
427     """Ставит/снимает \\Flagged + наш цветной keyword-флаг.

429     Gmail принимает произвольные keyword-флаги без вопросов, но
430     реальный корпоративный сервер (обнаружено на VK Mail) отвечает
431     "BAD [PARSE] Unable to parse flag" на STORE с несколькими нашими
432     keyword'ами разом — по всей видимости, сервер не разрешает
433     произвольные (не объявленные в PERMANENTFLAGS) keyword-флаги
434     вообще. Поэтому: (1) снимаем/ставим keyword'ы по одному, а не
435     разом — один отклонённый не должен мешать остальным; (2) если
436     сервер в принципе не принимает цветной keyword, тихо откатываемся
437     на стандартный \\Flagged, чтобы разметка не ломалась полностью
438     из-за того, что сервер не умеет в цвета.

440     `previous_color` — если вызывающий код уже знает текущий маркер
441     (обычно да — он же его и показывает в таблице), снимаем ТОЛЬКО
442     этот один keyword вместо того, чтобы вслепую пытаться снять все
443     6 возможных цветов на каждую смену маркера. На реальном сервере с
444     заметной сетевой задержкой это была разница между 1-2 round trip'ами
445     и 6 (жалоба: "маркер на письмах устанавливается очень долго").
446     Если previous_color не передан — поведение как раньше (безопасный,
447     но медленный вариант "снять всё возможное")."""
448     if previous_color is not UNKNOWN_MARKER and previous_color == color:
449         return # уже в нужном состоянии — нечего менять, даже SELECT не нужен
450     self._select(folder)
451     all_keywords = list(MARKER_COLORS.values())
452     # `color`/`previous_color` — один цвет или несколько через запятую
453     # (см. split_markers): на письме может быть несколько маркеров.
454     new_keywords = [MARKER_COLORS[c] for c in split_markers(color) if c in MARKER_COLORS]
455     if previous_color is UNKNOWN_MARKER:
456         to_remove = list(all_keywords)
457     elif previous_color is None:
458         to_remove = [] # маркера не было — снимать нечего, кроме самого нового keyword'а ниже не нужно
459     else:
460         to_remove = [MARKER_COLORS[c] for c in split_markers(previous_color) if c in MARKER_COLORS]

462     if not new_keywords:
463         self._remove_flags_best_effort(uid, [b"\\Flagged", *to_remove])
464         return
465     self._remove_flags_best_effort(uid, [k for k in to_remove if k not in new_keywords])
466     try:
467         self._client.add_flags([uid], [b"\\Flagged", *new_keywords])
468     except IMAPClientError:
469         self._client.add_flags([uid], [b"\\Flagged"])

471 def _remove_flags_best_effort(self, uid: int, flags: list[bytes]) -> None:
472     for flag in flags:
473         try:
474             self._client.remove_flags([uid], [flag])
475         except IMAPClientError:
476             pass # флаг и так не поддерживается/не был установлен — не критично

478 @_reconnecting
479 def move_messages(self, folder: str, uids: list[int], target_folder: str) -> None:
480     """Переносит письма в другую папку (например, в корзину) — атомарно,
481     если сервер поддерживает MOVE (RFC 6851), иначе COPY + удаление.

483     Если целевой папки ещё нет на сервере — создаёт её и повторяет
484     один раз, вместо того чтобы падать с сырым "[TRYCREATE] Folder
485     does not exist" (жалоба на правила сортировки почты: правило
486     может ссылаться на папку, которую пользователь только собирался
487     создать, а не создал заранее руками)."""
488     if not uids:
489         return
490     self._select(folder)

```

```

491         try:
492             self._move_or_copy(uids, target_folder)
493         except IMAPClientError as exc:
494             if "TRYCREATE" not in str(exc).upper():
495                 raise
496             self._client.create_folder(target_folder)
497             self._move_or_copy(uids, target_folder)

499     def _move_or_copy(self, uids: list[int], target_folder: str) -> None:
500         if self._client.has_capability("MOVE"):
501             self._client.move(uids, target_folder)
502         else:
503             self._client.copy(uids, target_folder)
504             self._client.delete_messages(uids)
505             self._client.expunge()

507     @_reconnecting
508     def delete_messages(self, folder: str, uids: list[int]) -> None:
509         """Безвозвратное удаление (Shift+Удалить, либо удаление из самой корзины)."""
510         if not uids:
511             return
512         self._select(folder)
513         self._client.delete_messages(uids)
514         self._client.expunge()

516     def _select(self, folder: str) -> None:
517         # Папка уже открыта этой же сессией – второй SELECT только теряет время.
518         if self._selected_folder != folder:
519             self._client.select_folder(folder, readonly=False)
520             self._selected_folder = folder

523     def _iter_body_parts(message: Message):
524         """Как Message.walk(), но НЕ спускается внутрь message/rfc822
525         (пересланное письмо целиком как вложение) – Message.walk() у стандартной
526         библиотеки считает такую часть "multipart" (её единственный payload –
527         вложенный Message-объект) и лезет внутрь неё саму же, вместо того
528         чтобы отдать её как один цельный узел вызывающему коду."""
529         if message.get_content_type() == "message/rfc822":
530             yield message
531             return
532         if message.is_multipart():
533             for part in message.get_payload():
534                 yield from _iter_body_parts(part)
535         else:
536             yield message

539     def extract_content(message: Message) -> MessageContent:
540         # Заголовки живут на верхнем уровне сообщения независимо от того,
541         # multipart оно или нет – читаем их один раз, а не в каждой из веток
542         # ниже (жалоба: "при просмотре письма невидно его реквизитов"), и
543         # передаём во все ветки через один хелпер, а не повторяя 5 kwargs в
544         # каждом return.
545         header_kwargs = dict(
546             subject=_decode_header_text(message.get("Subject")),
547             from=_decode_header_text(message.get("From")),
548             to=_decode_header_text(message.get("To")),
549             cc=_decode_header_text(message.get("Cc")),
550             bcc=_decode_header_text(message.get("Bcc")),
551         )

553         if not message.is_multipart():
554             if message.get_content_type() == "text/plain":
555                 # "or" – не просто "если текста вообще не было" (None), но и
556                 # "если он декодировался в пустую строку" (например,
557                 # get_payload(decode=True) не осилил конкретную кодировку и
558                 # вернул None → _decode_payload даёт "") – иначе панель чтения
559                 # оставалась молча пустой безо всякой подсказки, хотя реальный
560                 # веб-интерфейс почты то же письмо показывал с текстом (жалоба:
561                 # "некоторые письма открываются так [пусто], а на самом деле
562                 # они такие [с текстом]").
563                 return MessageContent(text=_decode_payload(message) or "(нет текстового содержимого)", **header_kwargs)
564             if message.get_content_type() == "text/html":
565                 return MessageContent(
566                     text="(письмо в формате HTML – предпросмотр текста недоступен)",
567                     html=_decode_payload(message),
568                     **header_kwargs,
569                 )
570             if message.get_content_type() == "text/calendar":
571                 return MessageContent(text="", attachments=[_calendar_attachment(message)], **header_kwargs)
572             return MessageContent(
573                 text="(письмо в формате HTML – предпросмотр текста недоступен)", **header_kwargs
574             )

576     text: str | None = None
577     html: str | None = None
578     attachments: list[Attachment] = []
579     inline_images: dict[str, tuple[str, bytes]] = {}

```



```

581     for part in _iter_body_parts(message):
582         if part.get_content_type() == "message/rfc822":
583             # Пересланное письмо целиком как вложение (Outlook и другие
584             # клиенты часто пересылают именно так, без явного
585             # Content-Disposition: attachment на этой части) – раньше
586             # message.walk() спускался ВНУТРЬ него (Python считает
587             # message/rfc822 "multipart", раз её единственный payload –
588             # вложенный Message), и текст/HTML пересылаемого письма
589             # помещивался в тело исходного вместо того, чтобы стать одним
590             # отдельным вложением – жалоба: "у письма есть вложение, но
591             # его нет в просмотре и при открытии".
592             nested = part.get_payload(0)
593             nested_subject = _decode_header_text(nested.get("Subject")) if nested else ""
594             filename = _decode_filename(part.get_filename()) or f"{nested_subject or 'Пересланное письмо'}.eml"
595             attachments.append(
596                 Attachment(
597                     filename=filename,
598                     content_type="message/rfc822",
599                     payload=nested.as_bytes() if nested else (part.get_payload(decode=True) or b""),
600                 )
601             )
602             continue

604         filename = _decode_filename(part.get_filename())
605         content_type = part.get_content_type()
606         content_disposition = part.get_content_disposition() # 'attachment' | 'inline' | None
607         content_id = (part.get("Content-Id") or "").strip().strip("<>")

609         # Картинка со своим Content-Id – то, на что ссылается  в HTML-теле, а не отдельное вложение для скачивания
611         # (даже если у неё есть имя файла и/или Content-Disposition).
612         if content_id and content_type.startswith("image/"):
613             inline_images[content_id] = (content_type, part.get_payload(decode=True) or b"")
614             continue

616         # text/plain и text/html – кандидаты в само тело письма, а не во
617         # вложение, даже если у части задан Content-Type: ...; name="..."
618         # (part.get_filename() читает и его, не только
619         # Content-Disposition: filename=) – реальные HTML-рассылки
620         # (например, от Авито) так подписывают HTML-часть письма без
621         # всякого намерения сделать её вложением. Раньше bool(filename)
622         # ниже срабатывал именно на этом и уводил всё письмо во вложение,
623         # оставляя тело пустым (жалоба: "письма в формате html не
624         # просматриваются"). Вложением текстовая часть считается только
625         # при явном Content-Disposition: attachment.
626         is_body_part = content_type in ("text/plain", "text/html") and content_disposition != "attachment"

628         # text/calendar (RFC 5546 iTIP-приглашение) сохраняем как вложение
629         # всегда – не только когда отправитель явно проставил
630         # Content-Disposition: attachment/filename (не все серверы это
631         # делают), иначе приглашение молча потеряется.
632         is_attachment = not is_body_part and (
633             bool(filename) or content_disposition == "attachment" or content_type == "text/calendar"
634         )
635         if is_attachment:
636             attachments.append(
637                 _calendar_attachment(part) if content_type == "text/calendar" else Attachment(
638                     filename=filename or "(без имени)",
639                     content_type=content_type,
640                     payload=part.get_payload(decode=True) or b"",
641                 )
642             )
643             continue

645         if content_type == "text/plain" and text is None:
646             text = _decode_payload(part)
647         elif content_type == "text/html" and html is None:
648             html = _decode_payload(part)

650     if not text:
651         # "not text", а не "text is None" – часть text/plain могла найтись,
652         # но не осилить декодирование (get_payload(decode=True) вернул
653         # None → _decode_payload дала "") и оставить панель чтения молча
654         # пустой без единой подсказки (жалоба: "некоторые письма
655         # открываются так [пусто], а на самом деле они такие [с текстом]").
656         text = "(письмо в формате HTML – предпросмотр текста недоступен)" if html else "(нет текстового
содержимого)"

658     return MessageContent(
659         text=text,
660         attachments=attachments,
661         html=html or "",
662         inline_images=inline_images,
663         **header_kwargs,
664     )

667 def _calendar_attachment(part: Message) -> Attachment:

```

```

668     return Attachment(
669         filename=_decode_filename(part.get_filename()) or "invite.ics",
670         content_type="text/calendar",
671         payload=part.get_payload(decode=True) or b"",
672     )

675 def _decode_header_text(value: str | None) -> str:
676     """Как _decode_filename, но для обычных текстовых заголовков (Subject/
677     From/To/Cc) – не все сервера сворачивают RFC 2047 encoded-word до
678     ENVELOPE, который парсит imapclient (см. _decode_subject); здесь тот
679     же случай, но для содержимого письма, разбираемого через email.message."""
680     if not value or "=" not in value:
681         return value or ""
682     return _decode_rfc2047(value.encode("ascii", errors="replace"))

685 def _decode_filename(filename: str | None) -> str | None:
686     """get_filename() отдаёт значение как есть – некоторые сервера (в т.ч.
687     встречались от Exchange) кодируют имя файла в RFC 2047 (=?utf-8?B?...?=)
688     вместо RFC 2231, которое email.message понимает само. Без декодирования
689     имя вложения показывалось бы пользователю нечитаемой закодированной
690     строкой вместо настоящего имени файла."""
691     if not filename or "=" not in filename:
692         return filename
693     return _decode_rfc2047(filename.encode("ascii", errors="replace"))

696 def _decode_payload(part: Message) -> str:
697     payload = part.get_payload(decode=True) or b""
698     charset = part.get_content_charset() or "utf-8"
699     return payload.decode(charset, errors="replace")

702 def _to_summary(data: dict) -> MessageSummary:
703     envelope = data[b"ENVELOPE"]
704     sender_display, sender_email = _format_address(envelope.from_)
705     message_id = envelope.message_id
706     flags = data.get(b"FLAGS", ())
707     marker_color = join_markers(_COLOR_BY_KEYWORD[f] for f in flags if f in _COLOR_BY_KEYWORD)
708     if marker_color is None and b"\\Flagged" in flags:
709         # Жалоба: "не сохраняется проставленный маркер, через какое-то
710         # время пропадает" – сервер (VK Mail) не хранит произвольные
711         # keyword-флаги ($RedMailRed и т.п.: их нет в PERMANENTFLAGS),
712         # принимает их только на время сессии и молча теряет, а стандартный
713         # \\Flagged хранит. Раньше без keyword'a маркер считался снятым –
714         # теперь \\Flagged без цвета = маркер по умолчанию; конкретный цвет
715         # при этом восстанавливается из локального кэша (см.
716         # CachedMailbox.refresh_folder).
717         marker_color = "red"
718     return MessageSummary(
719         uid=data[b"UID"],
720         subject=_decode_subject(envelope.subject),
721         sender=sender_display,
722         sender_email=sender_email,
723         date=envelope.date.strftime("%Y-%m-%d %H:%M") if envelope.date else "",
724         message_id=message_id.decode("ascii", errors="replace") if message_id else "",
725         has_attachments=_body_has_attachment(data[b"BODYSTRUCTURE"]) if b"BODYSTRUCTURE" in data else False,
726         marker_color=marker_color,
727         importance=parse_importance(message_from_bytes(data.get(b"BODY[HEADER.FIELDS (IMPORTANCE X-PRIORITY)]",
b""))),
728         is_read=b"\\Seen" in flags,
729         to=_format_address_list(getattr(envelope, "to", None)),
730         is_answered=b"\\Answered" in flags,
731     )

734 def _body_has_attachment(structure) -> bool:
735     """Смотрит в BODYSTRUCTURE, не скачивая тело письма целиком.

737     BODYSTRUCTURE – сырая вложенная структура по RFC 3501 (см. imapclient
738     response_types.BodyData), без готового поля "это вложение". Ищем
739     Content-Disposition: attachment паттерн-мэтчингом (2-элементный кортеж
740     вида (b'ATTACHMENT', (...))), а не по фиксированному индексу – точная
741     позиция disposition в кортеже "плавает" в зависимости от типа part'a
742     (у text/* и message/rfc822 есть дополнительные поля перед ней).
743     """
744     if structure.is_multipart:
745         parts, rest = structure[0], structure[1:]
746         if _disposition_is_attachment(rest):
747             return True
748         return any(_body_has_attachment(part) for part in parts)
749     # message/rfc822 (пересланное письмо целиком) – общепринято вложение
750     # само по себе, независимо от явного Content-Disposition: attachment
751     # (многие клиенты, включая Outlook, его не выставляют на такой части) –
752     # без этого скрепка в списке писем не показывалась вовсе (жалоба:
753     # "у письма есть вложение, но его нет в просмотре и при открытии").
754     if len(structure) >= 2 and _field_upper(structure[0]) == "MESSAGE" and _field_upper(structure[1]) == "RFC822":
755         return True

```

```

756     return _disposition_is_attachment(structure)

759 def _field_upper(value) -> str:
760     if isinstance(value, bytes):
761         return value.decode("ascii", errors="replace").upper()
762     return str(value).upper()

765 def _disposition_is_attachment(fields) -> bool:
766     for value in fields:
767         if isinstance(value, (tuple, list)) and len(value) == 2 and isinstance(value[0], bytes):
768             if value[0].upper() == b"ATTACHMENT":
769                 return True
770     return False

773 def parse_importance(headers: Message) -> str:
774     """Классифицирует важность письма по заголовкам Importance/X-Priority.

776     Публичная — используется и для IMAP (заголовки приходят отдельным полем
777     FETCH), и для archive_store (заголовки уже есть в разобранном письме)."""
778     importance = (headers.get("Importance") or "").strip().lower()
779     if importance in ("high", "urgent"):
780         return "high"
781     if importance == "low":
782         return "low"
783     priority = (headers.get("X-Priority") or "").strip()
784     if priority[:1] in ("1", "2"):
785         return "high"
786     if priority[:1] in ("4", "5"):
787         return "low"
788     return "normal"

791 def _decode_subject(raw: bytes | None) -> str:
792     if not raw:
793         return "(без темы)"
794     return _decode_rfc2047(raw)

797 def _decode_rfc2047(raw: bytes) -> str:
798     # Имена отправителей и темы писем сервер отдаёт как есть — они бывают
799     # в кодированных словах RFC 2047 (=?utf-8?B?...?), а не только сырым UTF-8.
800     parts = decode_header(raw.decode("ascii", errors="replace"))
801     return "".join(
802         chunk.decode(encoding or "utf-8", errors="replace") if isinstance(chunk, bytes) else chunk
803         for chunk, encoding in parts
804     )

807 def _format_address_list(addresses) -> str:
808     """Все адреса списка (To/Cc), через запятую — используется только для
809     отображения в списке писем, не для машинного разбора (см.
810     _parse_recipient_list в main_window.py для этого)."""
811     if not addresses:
812         return ""
813     parts = []
814     for address in addresses:
815         mailbox = address.mailbox.decode("utf-8", errors="replace") if address.mailbox else ""
816         host = address.host.decode("utf-8", errors="replace") if address.host else ""
817         name = f"{mailbox}@{host}" if mailbox and host else mailbox
818         email = _decode_rfc2047(address.name) if address.name else ""
819         parts.append(f"{name} <{email}>" if name and email else (email or name))
820     return ", ".join(parts)

823 def _format_address(addresses) -> tuple[str, str]:
824     """Возвращает (отображаемое имя, email-адрес) первого адреса в списке."""
825     if not addresses:
826         return "(неизвестно)", ""
827     address = addresses[0]
828     mailbox = address.mailbox.decode("utf-8", errors="replace") if address.mailbox else ""
829     host = address.host.decode("utf-8", errors="replace") if address.host else ""
830     email = f"{mailbox}@{host}" if mailbox and host else mailbox
831     if address.name:
832         return _decode_rfc2047(address.name), email
833     return (email or "(неизвестно)", email)

```

==== src/redmail/mailbox.py (148 строк) =====

```

1 from __future__ import annotations

3 from pathlib import Path

5 from redmail import archive_store, cache_store
6 from redmail.imap_client import UNKNOWN_MARKER, Account, ImapSession, MessageContent, MessageSummary

9 class CachedMailbox:

```

```

10     """Читает через ImapSession, но сперва проверяет локальный кэш.
11
12     folder_summaries() — чистое чтение из кэша, без обращения к серверу,
13     КРОМЕ самого первого раза, когда для папки ещё нет вообще ничего
14     закешированного (переключение между уже открытыми папками сети не
15     трогает — не нужно спрашивать сервер на каждый клик).
16
17     refresh_folder() — явная проверка сервера: при первом открытии папки,
18     по периодическому таймеру и по кнопке «Обновить» в UI. SELECT (дёшево,
19     без сканирования) выполняется всегда, чтобы узнать текущее число писем;
20     если оно не изменилось — обходимся без похода за самими письмами.
21
22     Текст письма и вложения кэшируются один раз навсегда: после доставки
23     письмо не меняется, повторно скачивать нечего.
24     """
25
26     def __init__(self, session: ImapSession, account: Account):
27         self.session = session
28         self._account_key = f"{account.host}:{account.username}"
29
30     def folder_summaries(self, folder: str, limit: int = 50) -> list[MessageSummary]:
31         cached = cache_store.get_folder_summaries(self._account_key, folder)
32         if cached:
33             return cached[:limit]
34         return self.refresh_folder(folder, limit)
35
36     def refresh_folder(self, folder: str, limit: int = 50) -> list[MessageSummary]:
37         total = self.session.folder_message_count(folder)
38         cached_total = cache_store.get_folder_exists(self._account_key, folder)
39         if cached_total == total:
40             cached = cache_store.get_folder_summaries(self._account_key, folder)
41             if cached:
42                 return cached[:limit]
43             summaries = self.session.fetch_summaries(limit)
44             # Жалоба: "не сохраняется проставленный маркер, через какое-то время
45             # пропадает" — сервер (VK Mail) не хранит наши цветные keyword-флаги,
46             # только стандартный \Flagged. Сервер — источник правды о том, ЕСТЬ
47             # ли маркер (\Flagged: снят в другом клиенте — снят и здесь), а
48             # КАКОГО он цвета — помнит локальный кэш (set_marker пишет туда при
49             # каждой смене). Без этого шага save_folder_summaries затирал бы
50             # цвет из кэша серверным "красный по умолчанию" при каждом
51             # обновлении (см. imap_client._to_summary).
52             cached_colors = {
53                 s.uid: s.marker_color
54                 for s in cache_store.get_folder_summaries(self._account_key, folder)
55                 if s.marker_color
56             }
57             for summary in summaries:
58                 if summary.marker_color is not None and summary.uid in cached_colors:
59                     summary.marker_color = cached_colors[summary.uid]
60             cache_store.save_folder_summaries(self._account_key, folder, total, summaries)
61             return summaries
62
63     def message_content(self, folder: str, uid: int) -> MessageContent:
64         cached = cache_store.get_message_content(self._account_key, folder, uid)
65         if cached is not None:
66             return cached
67         content = self.session.fetch_message_content(folder, uid)
68         cache_store.save_message_content(self._account_key, folder, uid, content)
69         return content
70
71     def message_raw(self, folder: str, uid: int) -> bytes:
72         """Не кэшируется — нужен только для разового экспорта в архив."""
73         return self.session.fetch_message_raw(folder, uid)
74
75     def search_uids(self, folder: str, *, before=None) -> list[int]:
76         """Не кэшируется — используется только для массовой выгрузки папки
77         в архив (вся папка / всё до даты), где важна полная папка на
78         сервере, а не то, что сейчас в локальном кэше сводок."""
79         return self.session.search_uids(folder, before=before)
80
81     def set_marker(self, folder: str, uid: int, color: str | None, *, previous_color=UNKNOWN_MARKER) -> None:
82         self.session.set_marker(folder, uid, color, previous_color=previous_color)
83         cache_store.set_marker(self._account_key, folder, uid, color)
84
85     def set_read(self, folder: str, uid: int, read: bool) -> None:
86         self.session.set_read(folder, uid, read)
87         cache_store.set_read(self._account_key, folder, uid, read)
88
89     def set_answered(self, folder: str, uid: int) -> None:
90         self.session.set_answered(folder, uid)
91         cache_store.set_answered(self._account_key, folder, uid)
92
93     def move_to_trash(self, folder: str, uids: list[int], trash_folder: str) -> None:
94         self.move_to_folder(folder, uids, trash_folder)
95
96     def move_to_folder(self, folder: str, uids: list[int], target_folder: str) -> None:
97         """Общий переезд писем в любую папку (не только корзину) — так же
98         используется при ручном применении правил сортировки почты."""

```

```

99         self.session.move_messages(folder, uids, target_folder)
100        cache_store.delete_messages(self._account_key, folder, uids)

102    def delete_messages(self, folder: str, uids: list[int]) -> None:
103        self.session.delete_messages(folder, uids)
104        cache_store.delete_messages(self._account_key, folder, uids)

106    def close(self) -> None:
107        self.session.close()

109    def append_message(self, folder: str, raw: bytes, *, flags: tuple = ()) -> None:
110        # Кэш сам заметит новое письмо при следующем открытии папки – там
111        # refresh_folder() уже сравнивает EXISTS с закешированным числом
112        # писем и перечитывает при расхождении, отдельная инвалидация не
113        # нужна.
114        self.session.append_message(folder, raw, flags=flags)

117    class ArchiveSource:
118        """Тот же протокол, что у CachedMailbox (folder_summaries/refresh_folder/
119        message_content/set_marker/delete_messages/close), но поверх локального
120        файла архива – чтобы UI не знал разницы между «живой ящик» и «архив». """

122        def __init__(self, path: Path):
123            self.path = path

125        def folder_summaries(self, folder: str, limit: int = 50) -> list[MessageSummary]:
126            return archive_store.list_messages(self.path, folder)[:limit]

128        def refresh_folder(self, folder: str, limit: int = 50) -> list[MessageSummary]:
129            # Архив не меняется извне сам по себе – «обновить» просто перечитывает файл.
130            return self.folder_summaries(folder, limit)

132        def message_content(self, folder: str, uid: int) -> MessageContent:
133            return archive_store.get_message_content(self.path, uid)

135        def set_marker(self, folder: str, uid: int, color: str | None, *, previous_color=UNKNOWN_MARKER) -> None:
136            archive_store.set_marker(self.path, uid, color)

138        def set_read(self, folder: str, uid: int, read: bool) -> None:
139            pass # архив всегда "прочитан" (см. folder_summaries) – переключать нечего

141        def set_answered(self, folder: str, uid: int) -> None:
142            pass # архив – статичный снимок, отвечать "заново" на его письма нельзя

144        def delete_messages(self, folder: str, uids: list[int]) -> None:
145            archive_store.delete_messages(self.path, uids)

147        def close(self) -> None:
148            pass

```

==== src/redmail/smtp_client.py (184 строк) ====

```

1  from __future__ import annotations

3  import smtplib
4  from collections.abc import Callable
5  from dataclasses import dataclass, field
6  from email import policy
7  from email.message import EmailMessage
8  from email.utils import formatdate, make_msgid

10 # Жалоба: "ошибка отправки приходит именно если отправить из redmail, а
11 # при отправке из VK всё уходит без ошибок" – реальная причина не в
12 # запятой в поле "Кому" (тот путь уже фильтрует пустые адреса, см.
13 # _parse_recipient_list в main_window.py), а в том, что политика по
14 # умолчанию для EmailMessage – email.policy.default с cte_type="8bit":
15 # для любого текста с кириллицей (почти каждое письмо здесь) это даёт
16 # Content-Transfer-Encoding: 8bit. Но smtplib.SMTP.send_message() решает,
17 # слать ли ESMTP-параметр BODY=8BITIME, ТОЛЬКО по тому, содержат ли
18 # non-ASCII символы САМИ АДРЕСА конверта (envelope from/to) – адреса это
19 # обычные email на латинице, поэтому BODY=8BITIME не запрашивается,
20 # и письмо с 8-битным телом уходит без него: формальное нарушение RFC
21 # 6152, которое строгие корпоративные шлюзы контентной фильтрации вполне
22 # резонно отклоняют уже после приёма (SMTP error ... after end of data:
23 # 500 Message rejected) – то самое поведение, "уходит, но потом
24 # отбойник". cte_type="7bit" заставляет content manager всегда кодировать
25 # нелатинский текст через quoted-printable/base64 (7-битно чистые,
26 # универсально совместимые кодировки) вместо сырых 8-битных байт –
27 # независимо от того, что там на стороне сервера с 8BITIME.
28 _OUTGOING_POLICY = policy.default.clone(cte_type="7bit")

31 @dataclass
32 class SmtAccount:
33     host: str
34     username: str
35     password: str
36     port: int = 587
37     use_ssl: bool = False # True = неявный TLS (порт 465), False = STARTTLS (порт 587)

```

```

38     # "password" — обычный AUTH LOGIN/PLAIN; "kerberos" — SSO для сервера в
39     # домене, см. Account.auth_type в imap_client.py и gssapi_sasl.py.
40     auth_type: str = "password"
41     # См. Account.keytab_path/principal — тот же необязательный keytab.
42     keytab_path: str = ""
43     principal: str = ""

46 @dataclass
47 class OutgoingAttachment:
48     filename: str
49     content_type: str
50     payload: bytes
51     # Доп. параметры Content-Type — нужно для приглашений: MIME-параметр
52     # method= (RFC 5546) рядом с METHOD: внутри самого .ics — многие клиенты
53     # (Outlook в их числе) ищут именно внешний параметр, чтобы показать
54     # интерфейс "принять/отклонить", а не просто вложение.
55     content_type_params: dict[str, str] = field(default_factory=dict)

58 @dataclass
59 class OutgoingMessage:
60     sender: str
61     to: list[str]
62     subject: str
63     body: str
64     cc: list[str] = field(default_factory=list)
65     bcc: list[str] = field(default_factory=list)
66     in_reply_to: str | None = None
67     references: list[str] = field(default_factory=list)
68     attachments: list[OutgoingAttachment] = field(default_factory=list)
69     # Отредактированное форматирование/встроенные картинки из ComposeDialog
70     # (жалоба: "нет возможности вставить картинку... не даёт установить
71     # какие-либо шрифты"). None — письмо чисто текстовое, шлём как раньше;
72     # когда задано, `body` остаётся обычным текстовым fallback-содержимым
73     # (RFC 2046 multipart/alternative — получатели без поддержки HTML
74     # видят ero), а html_body — реальное форматированное содержимое.
75     html_body: str | None = None
76     # cid -> (content_type, payload) — картинки, вставленные прямо в текст
77     # письма (не файловые вложения), связываются с html_body через
78     #  так же, как их парсит imap_client.extract_content
79     # у входящей почты.
80     inline_images: dict[str, tuple[str, bytes]] = field(default_factory=dict)

83 def build_email_message(message: OutgoingMessage) -> EmailMessage:
84     """Общая сборка RFC 822 письма — используется и для реальной отправки
85     по SMTP, и для того, чтобы положить готовую копию письма прямо в
86     "Отправленные"/"Черновики" через IMAP APPEND (сервер не всегда сам
87     сохраняет копию исходящих — жалоба: "не отображается отправка почты,
88     не появляется в папке отправленные")."""
89     email_message = EmailMessage(policy=OUTGOING_POLICY)
90     email_message["From"] = message.sender
91     email_message["To"] = ", ".join(message.to)
92     if message.cc:
93         email_message["Cc"] = ", ".join(message.cc)
94     if message.bcc:
95         # smtplib.send_message() сам добавляет адреса из Bcc в список
96         # получателей SMTP-конверта и одновременно вырезает сам заголовок
97         # Bcc из фактически передаваемого письма (см. исходники smtplib) —
98         # получателям Bcc не виден друг друга и остальным получателям.
99         email_message["Bcc"] = ", ".join(message.bcc)
100     email_message["Subject"] = message.subject
101     email_message["Date"] = formatdate(localtime=True)
102     email_message["Message-Id"] = make_msgid()
103     if message.in_reply_to:
104         email_message["In-Reply-To"] = message.in_reply_to
105     email_message["References"] = " ".join([*message.references, message.in_reply_to])
106     email_message.set_content(message.body)
107     if message.html_body:
108         email_message.add_alternative(message.html_body, subtype="html")
109         if message.inline_images:
110             html_part = email_message.get_payload()[-1]
111             for cid, (content_type, payload) in message.inline_images.items():
112                 maintype, _, subtype = content_type.partition("/")
113                 html_part.add_related(payload, maintype=maintype or "image", subtype=subtype or "png",
cid=f"<{cid}>")

115     for attachment in message.attachments:
116         maintype, _, subtype = attachment.content_type.partition("/")
117         email_message.add_attachment(
118             attachment.payload,
119             maintype=maintype or "application",
120             subtype=subtype or "octet-stream",
121             filename=attachment.filename,
122             params=attachment.content_type_params or None,
123         )
124     return email_message

```

```

127 def _with_retry(operation: Callable[[], None]) -> None:
128     """Один повтор на свежем соединении при обрыве на любом этапе —
129     коннект, STARTTLS, аутентификация (в т.ч. GSSAPI) или сама отправка.
130     Раньше единичный обрыв TCP/TLS падал прямо в интерфейс сырым
131     "Connection unexpectedly closed" (smtplib.SMTPServerDisconnected,
132     который наследуется от OSError) — жалоба при отправке приглашения на
133     встречу: "Встреча сохранена, но не разослана". Тот же принцип, что
134     уже применён для IMAP в imap_client.py._reconnecting."""
135     try:
136         operation()
137     except (OSError, EOFError):
138         operation()

141 def _connect_and_authenticate(account: SmtplibAccount) -> smtplib.SMTP:
142     smtp_cls = smtplib.SMTP_SSL if account.use_ssl else smtplib.SMTP
143     client = smtp_cls(account.host, account.port, timeout=30)
144     if not account.use_ssl:
145         client.starttls()
146     if account.auth_type == "kerberos":
147         # Импорт внутри функции — см. imap_client.py._login: gssapi
148         # нужен только для SSO и не должен ломать обычный пароль там,
149         # где нет системных библиотек Kerberos.
150         from redmail import gssapi_sasl

152         gssapi_sasl.smtp_sasl_login(
153             client,
154             account.host,
155             account.username,
156             keytab_path=account.keytab_path,
157             principal=account.principal,
158         )
159     else:
160         client.login(account.username, account.password)
161     return client

164 def test_connection(account: SmtplibAccount) -> None:
165     """Подключается и проходит аутентификацию, ничего не отправляя — для
166     кнопки "Проверить подключение" в настройках (жалоба-пожелание:
167     "может добавить кнопку проверки подключения для входящих, исходящих
168     и календаря?"). Успех — соединение установлено и закрыто без ошибок;
169     любая проблема (сеть, TLS, логин/SSO) всплывает как исключение."""
170     def attempt() -> None:
171         with _connect_and_authenticate(account):
172             pass

174     _with_retry(attempt)

177 def send_message(account: SmtplibAccount, message: OutgoingMessage) -> None:
178     email_message = build_email_message(message)

180     def attempt() -> None:
181         with _connect_and_authenticate(account) as client:
182             client.send_message(email_message)

184     _with_retry(attempt)

```

==== src/redmail/itip.py (301 строк) ====

```

1 from __future__ import annotations

3 import base64
4 from dataclasses import dataclass, replace
5 from datetime import date, datetime, timezone
6 from email.message import Message
7 from pathlib import Path

9 import icalendar

11 from redmail import calendar_store
12 from redmail.calendar_store import Attendee, Event, new_uid
13 from redmail.imap_client import Attachment

15 # iTIP (RFC 5546) поверх обычной почты — единственный способ приглашений/
16 # ответов/переносов, который одинаково работает через Exchange, VK Mail и
17 # любой другой сервер по IMAP/SMTP: сам .ics-формат встраивания в письмо
18 # (text/calendar, METHOD:REQUEST/REPLY/CANCEL) — общий стандарт, которому
19 # следует и Outlook, а не что-то специфичное для одного сервера.

21 _PARTSTAT_TO_LOCAL = {
22     "ACCEPTED": "accepted",
23     "DECLINED": "declined",
24     "TENTATIVE": "tentative",
25     "NEEDS-ACTION": "needs-action",
26 }
27 _PARTSTAT_TO_ICAL = {v: k for k, v in _PARTSTAT_TO_LOCAL.items()}

```

```

30 @dataclass
31 class IncomingInvite:
32     method: str # "REQUEST" | "REPLY" | "CANCEL" | ...
33     event: Event
34     replying_attendee_email: str = "" # заполнено только для method == "REPLY"

37 class NotAnInviteError(Exception):
38     """В письме нет разбираемого text/calendar с VEVENT."""

41 def find_calendar_part(message: Message) -> bytes | None:
42     """Ищет вложенный .ics (text/calendar - так его встраивает любой iTIP-
43     совместимый отправитель: Outlook/Exchange, VK Mail, сам redmail) в уже
44     разобранном письме."""
45     if message.get_content_type() == "text/calendar":
46         return message.get_payload(decode=True)
47     if message.is_multipart():
48         for part in message.walk():
49             if part.get_content_type() == "text/calendar":
50                 return part.get_payload(decode=True)
51     return None

54 def parse_invite(ics_bytes: bytes, my_email: str) -> IncomingInvite:
55     cal = icalendar.Calendar.from_ical(ics_bytes)
56     method = str(cal.get("method", "REQUEST")).upper()

58     vevent = None
59     for component in cal.walk():
60         if component.name == "VEVENT":
61             vevent = component
62             break
63     if vevent is None:
64         raise NotAnInviteError("В .ics нет VEVENT")

66     event = _event_from_vevent(vevent, method, my_email, ics_bytes)
67     replying_attendee_email = event.attendees[0].email if method == "REPLY" and event.attendees else ""
68     return IncomingInvite(method=method, event=event, replying_attendee_email=replying_attendee_email)

71 def parse_ics_events(ics_bytes: bytes, my_email: str) -> list[Event]:
72     """Разбирает ВСЕ VEVENT из .ics - экспорт целого календаря (несколько
73     встреч без METHOD:, в отличие от одиночного приглашения), например
74     файл, выгруженный из VK Mail/Google/Outlook через "Экспорт календаря".
75     Записи, которые не удалось разобрать (битые/неполные), пропускаются -
76     одна плохая запись не должна срывать импорт всего файла."""
77     cal = icalendar.Calendar.from_ical(ics_bytes)
78     method = str(cal.get("method", "PUBLISH")).upper()
79     events: list[Event] = []
80     for component in cal.walk():
81         if component.name != "VEVENT":
82             continue
83         try:
84             events.append(_event_from_vevent(component, method, my_email, None))
85         except (KeyError, ValueError):
86             continue
87     return events

90 def import_ics(path: Path, ics_bytes: bytes, my_email: str = "", calendar_id: str | None = None) -> int:
91     """Импортирует все события из .ics-файла (выгрузка целого календаря
92     внешней системой - например, "Экспорт" из VK Mail/Google/Outlook, а
93     не одиночное приглашение) в локальный calendar_store.

94     calendar_id - куда положить импортированные события. Раньше не
95     передавался вовсе, и события всегда попадали в календарь по умолчанию,
96     смешиваясь с личными встречами - не было способа получить именно
97     "календарь из другого источника" отдельным списком со своим
98     цветом/видимостью (жалоба: "не создаётся календарь из другого
99     источника"). None - прежнее поведение (calendar_store.DEFAULT_CALENDAR_ID).
100     """
101     events = parse_ics_events(ics_bytes, my_email)
102     if calendar_id is not None:
103         events = [replace(event, calendar_id=calendar_id) for event in events]
104     for event in events:
105         calendar_store.save_event(path, event)
106     return len(events)

110 def _event_from_vevent(vevent, method: str, my_email: str, raw_ics: bytes | None) -> Event:
111     dtstart_value = vevent["DTSTART"].dt
112     all_day = not isinstance(dtstart_value, datetime) and isinstance(dtstart_value, date)
113     dtstart = _to_utc(dtstart_value)
114     dtend_prop = vevent.get("DTEND")
115     dtend = _to_utc(dtend_prop.dt) if dtend_prop is not None else dtstart

117     organizer_prop = vevent.get("ORGANIZER")

```



```

118     organizer_email = _address_email(organizer_prop) if organizer_prop else ""
119     organizer_name = _address_name(organizer_prop) if organizer_prop else ""

121     attendees: list[Attendee] = []
122     my_participation = "needs-action"
123     for prop in _as_list(vevent.get("ATTENDEE")):
124         email_addr = _address_email(prop)
125         partstat = _PARTSTAT_TO_LOCAL.get(str(prop.params.get("PARTSTAT", "NEEDS-ACTION")).upper(), "needs-action")
126         attendees.append(Attendee(email=email_addr, name=_address_name(prop), participation=partstat))
127         if my_email and email_addr.lower() == my_email.lower():
128             my_participation = partstat

130     rrule_prop = vevent.get("RRULE")
131     recurrence_rule = rrule_prop.to_ical().decode("ascii") if rrule_prop is not None else None

133     attachments = [
134         a for i, prop in enumerate(_as_list(vevent.get("ATTACH"))) if (a := _attachment_from_prop(prop, i))
135     ]

137     uid = str(vevent.get("UID", "")) or new_uid()

139     # Реальный экспорт целого календаря (в отличие от одиночного iTIP-
140     # приглашения) не несёт METHOD:, зато почти всегда несёт свой STATUS:
141     # на каждой встрече (в реальном файле VK Mail – 154 из 174 событий
142     # TENTATIVE) – раньше это вообще не читалось, и все такие события
143     # тихо превращались в "confirmed".
144     status_prop = str(vevent.get("STATUS", "")).upper()
145     if method == "CANCEL" or status_prop == "CANCELLED":
146         status = "cancelled"
147     elif status_prop == "TENTATIVE":
148         status = "tentative"
149     else:
150         status = "confirmed"

152     return Event(
153         uid=uid,
154         sequence=int(vevent.get("SEQUENCE", 0)),
155         summary=str(vevent.get("SUMMARY", "")),
156         description=str(vevent.get("DESCRIPTION", "")),
157         location=str(vevent.get("LOCATION", "")),
158         dtstart=dtstart,
159         dtend=dtend,
160         all_day=all_day,
161         recurrence_rule=recurrence_rule,
162         organizer_email=organizer_email,
163         organizer_name=organizer_name,
164         is_organizer=bool(my_email) and bool(organizer_email) and organizer_email.lower() == my_email.lower(),
165         status=status,
166         my_participation=my_participation,
167         attendees=attendees,
168         attachments=attachments,
169         raw_ics=raw_ics,
170     )

173 def build_request_ics(event: Event, organizer_email: str, organizer_name: str) -> bytes:
174     return _build(
175         "REQUEST", event, organizer_email=organizer_email, organizer_name=organizer_name, status="CONFIRMED"
176     )

179 def build_cancel_ics(event: Event, organizer_email: str, organizer_name: str) -> bytes:
180     return _build(
181         "CANCEL", event, organizer_email=organizer_email, organizer_name=organizer_name, status="CANCELLED"
182     )

185 def build_caldav_ics(event: Event, organizer_email: str, organizer_name: str) -> bytes:
186     """Объект календаря для PUT на CalDAV-сервер – это не приглашение по
187     почте (METHOD:REQUEST/CANCEL из RFC 5546), а обычное хранимое событие,
188     поэтому без верхнеуровневого iTIP METHOD (см. RFC 4791 – CalDAV сам по
189     себе не про METHOD, это только для scheduling-сообщений)."""
190     return _build(None, event, organizer_email=organizer_email, organizer_name=organizer_name,
status=event.status.upper())

193 def build_reply_ics(event: Event, attendee_email: str, attendee_name: str, participation: str) -> bytes:
194     cal = _new_calendar("REPLY")
195     vevent = iCalendar.Event()
196     vevent.add("uid", event.uid)
197     vevent.add("summary", event.summary)
198     vevent.add("sequence", event.sequence)
199     vevent.add("dtstamp", datetime.now(timezone.utc))
200     vevent.add("dtstart", event.dtstart.date() if event.all_day else event.dtstart)

202     organizer = iCalendar.vCalAddress(f"mailto:{event.organizer_email}")
203     if event.organizer_name:
204         organizer.params["CN"] = event.organizer_name
205     vevent.add("organizer", organizer, encode=0)

```

```

207     attendee = icalendar.vCalAddress(f"mailto:{attendee_email}")
208     attendee.params["CN"] = attendee_name or attendee_email
209     attendee.params["PARTSTAT"] = _PARTSTAT_TO_ICAL.get(participation, "NEEDS-ACTION")
210     vevent.add("attendee", attendee, encode=0)

212     cal.add_component(vevent)
213     return cal.to_ical()

216 def _build(method: str | None, event: Event, *, organizer_email: str, organizer_name: str, status: str) -> bytes:
217     cal = _new_calendar(method)
218     vevent = icalendar.Event()
219     vevent.add("uid", event.uid)
220     vevent.add("summary", event.summary)
221     if event.description:
222         vevent.add("description", event.description)
223     if event.location:
224         vevent.add("location", event.location)
225     vevent.add("sequence", event.sequence)
226     vevent.add("status", status)
227     vevent.add("dtstamp", datetime.now(timezone.utc))
228     if event.all_day:
229         vevent.add("dtstart", event.dtstart.date())
230         vevent.add("dtend", event.dtend.date())
231     else:
232         vevent.add("dtstart", event.dtstart)
233         vevent.add("dtend", event.dtend)
234     if event.recurrence_rule:
235         vevent.add("rrule", icalendar.vRecur.from_ical(event.recurrence_rule))

237     organizer = icalendar.vCalAddress(f"mailto:{organizer_email}")
238     organizer.params["CN"] = organizer_name or organizer_email
239     vevent.add("organizer", organizer, encode=0)

241     for attendee in event.attendees:
242         addr = icalendar.vCalAddress(f"mailto:{attendee.email}")
243         addr.params["CN"] = attendee.name or attendee.email
244         addr.params["ROLE"] = "REQ-PARTICIPANT"
245         addr.params["PARTSTAT"] = _PARTSTAT_TO_ICAL.get(attendee.participation, "NEEDS-ACTION")
246         addr.params["RSVP"] = "TRUE" if method == "REQUEST" else "FALSE"
247         vevent.add("attendee", addr, encode=0)

249     for attachment in event.attachments:
250         attach_prop = icalendar.vBinary(attachment.payload)
251         attach_prop.params["FMTTYPE"] = attachment.content_type
252         attach_prop.params["X-FILENAME"] = attachment.filename
253         vevent.add("attach", attach_prop, encode=0)

255     cal.add_component(vevent)
256     return cal.to_ical()

259 def _new_calendar(method: str | None) -> icalendar.Calendar:
260     cal = icalendar.Calendar()
261     cal.add("prodid", "-//RedMail//RU")
262     cal.add("version", "2.0")
263     if method:
264         cal.add("method", method)
265     return cal

268 def _as_list(prop) -> list:
269     if prop is None:
270         return []
271     return prop if isinstance(prop, list) else [prop]

274 def _address_email(prop) -> str:
275     value = str(prop)
276     return value[7:] if value.lower().startswith("mailto:") else value

279 def _address_name(prop) -> str:
280     return str(prop.params.get("CN", "")) if hasattr(prop, "params") else ""

283 def _attachment_from_prop(prop, index: int) -> Attachment | None:
284     # ATTACH бывает и внешней ссылкой (URI), а не встроенным файлом — тогда
285     # icalendar отдаёт не vBinary, а обычную строку/vUri; такие пропускаем
286     # (нечего скачивать/показывать как вложение без похода в сеть).
287     if not isinstance(prop, icalendar.vBinary):
288         return None
289     try:
290         payload = base64.b64decode(prop.to_ical())
291     except (ValueError, TypeError):
292         return None
293     filename = str(prop.params.get("X-FILENAME") or prop.params.get("FILENAME") or f"attachment-{index + 1}")
294     content_type = str(prop.params.get("FMTTYPE", "application/octet-stream"))

```

```

295         return Attachment(filename=filename, content_type=content_type, payload=payload)

298 def _to_utc(value) -> datetime:
299     if isinstance(value, datetime):
300         return value.astimezone(timezone.utc) if value.tzinfo else value.replace(tzinfo=timezone.utc)
301     return datetime(value.year, value.month, value.day, tzinfo=timezone.utc)

==== src/redmail/caldav_sync.py (536 строк) ====
1  from __future__ import annotations

3  import urllib.error
4  import urllib.request
5  from dataclasses import dataclass
6  from datetime import datetime, timedelta, timezone
7  from urllib.parse import unquote, urlparse
8  from uuid import uuid4

10 import caldav
11 import requests
12 from caldav.lib.error import AuthorizationError, NotFoundError

14 from redmail import itip
15 from redmail.calendar_store import Event

17 # Некоторые корпоративные прокси/WAF перед CalDAV-сервером отличают
18 # автоматизированные HTTP-библиотеки от обычных десктопных клиентов именно
19 # по User-Agent и применяют к ним более строгую политику — жалоба
20 # "из Evolution и через веб событие создаётся нормально, а из этого клиента
21 # нет, хотя проверка подключения (чтение) проходит" указывает ровно на такую
22 # избирательную фильтрацию по заголовку, а не на блокировку метода PUT как
23 # такового (раз PUT работает у других клиентов на этом же сервере). Библиотека
24 # caldav по умолчанию подставляет "python-caldav/<версия>" — максимально
25 # узнаваемую сигнатуру скриптовой библиотеки; заменяем её на нейтральную,
26 # не выдающую, что это python-скрипт, но и не выдающую себя за чужой продукт.
27 _CALDAV_USER_AGENT = "redmail-caldav-client/1.0"

30 def _with_connection_retry(func, *args, **kwargs):
31     """Один повтор САМОГО сетевого вызова при ConnectionError ("Remote end
32     closed connection without response" и т.п.) — жалоба: "не
33     синхронизируется календарь, при этом проверка подключения проходит".
34     Проверка подключения — это единственный лёгкий PROPFIND, а сама
35     синхронизация — несколько последовательных запросов (push каждой
36     встречи, потом fetch) через пул соединений requests/caldav; сервер
37     (или прокси/балансировщик перед ним в закрытой корпоративной сети)
38     вполне может закрыть простаивающее keep-alive-соединение между
39     запросами, не ответив на очередной — классический случай устаревшего
40     соединения, который в подавляющем большинстве случаев лечится одним
41     повтором на свежем соединении, а не признак настоящей неполадки с
42     сервером или данными. Обёрнуто вокруг КОНКРЕТНОГО вызова caldav/requests,
43     а не всего метода целиком — иначе к моменту, когда метод сам ловит
44     Exception и заворачивает его в CalDavSyncError, исходный
45     requests.exceptions.ConnectionError уже не различить."""
46     try:
47         return func(*args, **kwargs)
48     except requests.exceptions.ConnectionError:
49         return func(*args, **kwargs)

51 # CalDAV с сервером клиента (VK Mail/Exchange) — сеть закрытая корпоративная,
52 # у самого redmail нет прямого способа её нащупать заранее, поэтому адрес
53 # сервера вводится пользователем вручную в настройках (см. SettingsDialog/
54 # config_store.load_caldav_settings), а не автоопределяется. Логин/пароль —
55 # те же, что для IMAP/SMTP (общий Account), см. MainWindow.on_caldav_sync.
56 #
57 # Двусторонняя синхронизация: события, где мы организатор, отправляются на
58 # сервер (push); всё, что есть на сервере в окне синхронизации, подтягивается
59 # в локальный calendar_store (pull) и связывается по UID (тот же UID, что
60 # использует iTIP-путь через почту — см. itip.py) — событие, полученное
61 # когда-то по приглашению, и то же событие с CalDAV-сервера не задвоятся.

64 @dataclass
65 class CalDavAccount:
66     url: str
67     username: str
68     password: str
69     # "password" — Basic (логин + app-пароль); "kerberos" — SSO по доменному
70     # билету через SPNEGO/Negotiate, как у IMAP/SMTP (Account.auth_type).
71     # Уточнение от пользователя: веб-приложение VK авторизует через keytab,
72     # то есть HTTP-фронт (тот же, что отдаёт CalDAV) принимает Negotiate —
73     # тогда app-пароль (который VK для сторонних клиентов требует и порой
74     # капризно: "Application password is REQUIRED") не нужен вовсе.
75     auth_type: str = "password"
76     # См. imap_client.Account.keytab_path/principal — тот же необязательный
77     # keytab как источник билета для SSO.
78     keytab_path: str = ""
79     principal: str = ""

```

```

82 class CalDavSyncError(Exception):
83     """Ошибка подключения к серверу или синхронизации — показывается
84     пользователю как есть, не проplatывается молча (тот же принцип, что и
85     у остальных сетевых операций в проекте)."""

88 def probe_auth_schemes(url: str) -> list[str]:
89     """Делает запрос без учётных данных к CalDAV-серверу и смотрит, какие
90     схемы аутентификации он предлагает в ответе 401 (заголовок
91     WWW-Authenticate) — чтобы понять, есть ли там Kerberos/SPNEGO
92     (Negotiate), а не гадать вслепую по одному лишь "Unauthorized".
93     Реальный повод: тонкий веб-клиент (аналог OWA) у той же почты
94     заходит через SSO, значит Kerberos-инфраструктура в организации
95     есть — но CalDAV в этом приложении сейчас умеет только логин и
96     пароль (Basic), и неясно, предлагает ли сам CalDAV-сервер вообще
97     Negotiate, пока не проверишь напрямую."""
98     req = urllib.request.Request(url, method="PROPFIND", headers={"Depth": "0"})
99     try:
100         urllib.request.urlopen(req, timeout=10)
101         return [] # ответил без 401 вовсе — не тот случай, для которого зовут эту функцию
102     except urllib.error.HTTPError as exc:
103         if exc.code != 401:
104             return []
105         values = exc.headers.get_all("WWW-Authenticate") or []
106         return [v.split(None, 1)[0] for v in values if v.strip()]
107     except urllib.error.URLError:
108         return [] # диагностика best-effort — не должна маскировать исходную ошибку вызывающего кода

111 def _auth_scheme_hint(url: str) -> str:
112     schemes = probe_auth_schemes(url)
113     if not schemes:
114         return ""
115     if any(s.lower() == "negotiate" for s in schemes):
116         return (
117             f"Сервер предлагает: {' '.join(schemes)} — включая Negotiate (Kerberos/SPNEGO): "
118             "переключите способ входа учётной записи на Kerberos (SSO) — CalDAV тогда пойдёт "
119             "по доменному билету, без app-пароля."
120         )
121     return f"Сервер предлагает только: {' '.join(schemes)} — SSO (Negotiate) на этом сервере недоступен."

124 @dataclass
125 class CalDavCalendarInfo:
126     """Один календарь, обнаруженный на сервере при обходе calendar-home-set —
127     задача "настроить получение расшаренных календарей для VK Mail". У VK
128     (и вообще по CalDAV, см. переписку с пользователем) нет делегирования
129     всего аккаунта (calendar-проху, как у Apple/Nextcloud) — есть только
130     пошаренные по отдельности календари, и после того как коллега
131     расшарил календарь и приглашение принято В ВЕБ-ИНТЕРФЕЙСЕ VK (accept-
132     invite по CalDAV не бывает), сервер сам кладёт этот календарь в NAME
133     calendar-home-set как обычную коллекцию — единственное, что остаётся
134     сделать здесь, это её ОБНАРУЖИТЬ и показать, чей это календарь."""

136     url: str
137     name: str
138     owner: str | None
139     is_shared: bool
140     read_only: bool

143 _CALDAV_CALENDAR_TAG = "{urn:ietf:params:xml:ns:caldav}calendar"
144 _CALDAV_HOME_SET_PROP = "{urn:ietf:params:xml:ns:caldav}calendar-home-set"
145 _CALENDAR_DISCOVERY_PROPS = [
146     "{DAV:}resourcetype",
147     "{DAV:}displayname",
148     "{DAV:}owner",
149     "{DAV:}current-user-privilege-set",
150 ]

153 @dataclass
154 class PropfindResult:
155     """Один <response> из multistatus-ответа на PROPFIND: href, статус и
156     свойства (tag → значение: текст, список тегов для resourcetype, список
157     href для ссылочных свойств, сам элемент для сложных вроде
158     current-user-privilege-set)."""

160     href: str
161     status: int
162     properties: dict[str, object]

165 def _propfind_body(props: list[str]) -> str:
166     """Тело PROPFIND-запроса под нужные свойства. Библиотека caldav в
167     разных версиях по-разному принимает список свойств и по-разному
168     отдаёт разобранный ответ (в 3.x — results, в 2.x — только дерево);
169     свой XML на входе и свой разбор дерева на выходе (_parse_multistatus)

```

```

170     не зависят от этих различий — нужен только сырой lxml-tree ответа.""
171     from lxml import etree

173     root = etree.Element("{DAV:}propfind", nsmap={"d": "DAV:", "c": "urn:ietf:params:xml:ns:caldav"})
174     prop = etree.SubElement(root, "{DAV:}prop")
175     for name in props:
176         etree.SubElement(prop, name)
177     return etree.tostring(root, xml_declaration=True, encoding="utf-8").decode("utf-8")

180 def _status_code(text: str | None) -> int:
181     for part in (text or "").split():
182         if part.isdigit():
183             return int(part)
184     return 200

187 def _prop_value(element) -> object:
188     children = [child for child in element if isinstance(child.tag, str)]
189     if not children:
190         return (element.text or "").strip()
191     if element.tag == "{DAV:}resourcetype":
192         return [child.tag for child in children]
193     if all(child.tag == "{DAV:}href" for child in children):
194         return [(child.text or "").strip() for child in children]
195     return element

198 def _parse_multistatus(tree) -> list[_PropfindResult]:
199     """Разбор <multistatus> ответа PROPFIND (RFC 4918): по одному
200     результату на <response>, в properties — только свойства из
201     propstat со статусом 2xx."""
202     results: list[_PropfindResult] = []
203     if tree is None:
204         return results
205     for response in tree.iter("{DAV:}response"):
206         href_el = response.find("{DAV:}href")
207         href = (href_el.text or "").strip() if href_el is not None else ""
208         properties: dict[str, object] = {}
209         statuses: list[int] = []
210         for propstat in response.findall("{DAV:}propstat"):
211             status_el = propstat.find("{DAV:}status")
212             code = _status_code(status_el.text if status_el is not None else None)
213             statuses.append(code)
214             if code // 100 != 2:
215                 continue
216             prop = propstat.find("{DAV:}prop")
217             if prop is None:
218                 continue
219             for element in prop:
220                 if isinstance(element.tag, str):
221                     properties[element.tag] = _prop_value(element)
222         if statuses:
223             status = 200 if any(code // 100 == 2 for code in statuses) else statuses[0]
224         else:
225             direct = response.find("{DAV:}status")
226             status = _status_code(direct.text) if direct is not None else 200
227         results.append(_PropfindResult(href=href, status=status, properties=properties))
228     return results

231 def _href_path(href: str) -> str:
232     """Путь без схемы/хоста и завершающего слэша — owner может прийти и
233     абсолютным URL, и просто путём, сравнивать нужно только путь."""
234     return unquote(urlparse(href).path).rstrip("/")

237 def _has_write_privilege(value: object) -> bool:
238     """current-user-privilege-set не разбирается caldav-библиотекой в
239     удобный список (вложенные <privilege><write/></privilege> не подходят
240     под её общий разбор свойств) — приходит как есть, "сырым" XML-
241     элементом, ищем write/all прямо в нём. Свойство отсутствует вовсе
242     (None) — не считаем календарь урезанным без причины: сервер и так
243     отклонит PUT 403-м, если прав на самом деле нет, а свойство мог просто
244     не отдать (см. переписку: cs:invite/cs:shared-url у VK могут прийти
245     404 — не факт, что current-user-privilege-set при этом тоже не будет)."""
246     if value is None:
247         return True
248     if not hasattr(value, "iter"):
249         return False
250     for child in value.iter():
251         local = child.tag.rsplit(":", 1)[-1] if isinstance(child.tag, str) else ""
252         if local in ("write", "all", "write-content"):
253             return True
254     return False

257 class CalDavSession:
258     """Одно CalDAV-соединение на сессию синхронизации — тот же принцип, что

```

```

259 у ImapSession: подключение переиспользуется, а не открывается заново на
260 каждую операцию. Синхронизация запускается вручную (кнопка в
261 календаре), автоматического фонового опроса нет – сервер ещё ни разу
262 не проверялся вживую, включать это по таймеру было бы преждевременно."""

264 def __init__(self, account: CalDavAccount):
265     self.account = account
266     # Без явного таймаута зависший/недоступный сервер (закрытая
267     # корпоративная сеть, где угодно может быть неверно настроенный
268     # прокси/файрвол) мог держать HTTP-запрос сколько угодно – а весь
269     # on_caldav_sync() до сих пор шёл синхронно в основном потоке
270     # интерфейса: жалоба "после настройки CalDAV сломалась отправка,
271     # просмотр, переход между папками и получение почты" – на деле не
272     # сломалась, а всё это время буквально ждала одного зависшего
273     # сетевого запроса. См. MainWindow.on_caldav_sync – сама
274     # синхронизация теперь выполняется в фоновом потоке, но таймаут
275     # всё равно нужен: без него поток просто завис бы бесконечно вместо
276     # того, чтобы сообщить об ошибке.
277     client_kwargs: dict = {"timeout": 30, "headers": {"User-Agent": _CALDAV_USER_AGENT}}
278     try:
279         if account.auth_type == "kerberos":
280             # Импортирует внутри – см. gssapi_sasl.py: requests_gssapi тянет
281             # системные библиотеки Kerberos, которых нет там, где SSO не
282             # используется (и на машине для тестов); обычный пароль от
283             # этого ломаться не должен.
284             import requests_gssapi

286             from redmail import gssapi_sasl

288             # mutual_authentication=OPTIONAL, а не REQUIRED по умолчанию:
289             # за корпоративным reverse-проxy/балансировщиком ответ сервера
290             # часто приходит без встречного GSSAPI-токена, и строгая
291             # взаимная проверка роняла бы уже успешно прошедший вход.
292             # creds=None – билет из системного кэша; с keytab – из него.
293             client_kwargs["auth"] = requests_gssapi.HTTPSPNEGOAuth(
294                 mutual_authentication=requests_gssapi.OPTIONAL,
295                 creds=gssapi_sasl.acquire_credentials(account.keytab_path, account.principal),
296             )
297         else:
298             client_kwargs["username"] = account.username
299             client_kwargs["password"] = account.password
300             self._client = caldav.DAVClient(account.url, **client_kwargs)
301             # Всегда обычный requests.Session вместо того, что выбрал caldav:
302             # при наличии niquests он берёт его, а niquests по умолчанию
303             # договаривается об HTTP/2 через ALPN. Жалоба (и после смены
304             # User-Agent, и после прямого URL календаря): чтение работает,
305             # а PUT события рвётся "Remote end closed connection without
306             # response" – при этом Evolution (libsoup, HTTP/1.1) на том же
307             # сервере события создаёт. Обрыв запроса с телом без единого
308             # байта ответа при рабочих запросах без тела – характерный
309             # симптом прокси/балансировщика с неполной поддержкой HTTP/2
310             # для методов с телом; обычный requests ходит по HTTP/1.1, как
311             # Evolution. Заодно с ним документирована и отлажена связка с
312             # HTTPSPNEGOAuth (хук на 401 через r.connection/r.request.copy()).
313             try:
314                 self._client.session.close()
315             except Exception:
316                 pass
317             self._client.session = requests.Session()
318         except Exception as exc:
319             raise CalDavSyncError(f"Не удалось создать CalDAV-соединение: {exc}") from exc
320         self._calendar = None

322     def _primary_calendar(self):
323         if self._calendar is None:
324             # account.url – это URL КОНКРЕТНОГО календаря (см.
325             # list_calendars_detailed для обнаружения таких URL, включая
326             # расшаренные другими пользователями), обращаемся к нему
327             # напрямую. Раньше здесь было principal.calendars()[0] –
328             # дискавери первого попавшегося календаря аккаунта НЕЗАВИСИМО
329             # от того, какой именно caldav url был настроен: из-за этого
330             # ЛЮБОЙ второй настроенный CalDAV-календарь того же аккаунта
331             # молча синхронизировался бы с тем же самым первым календарём,
332             # что и делало поддержку нескольких/расшаренных календарей
333             # бессмысленной – без этого исправления задача "получение
334             # расшаренных календарей для VK Mail" попросту не работала бы.
335             self._calendar = caldav.Calendar(client=self._client, url=self.account.url)
336         return self._calendar

338     def list_calendar_names(self) -> list[str]:
339         try:
340             principal = self._client.principal()
341             return [cal.get_display_name() or str(cal.url) for cal in _with_connection_retry(principal.calendars)]
342         except AuthorizationError as exc:
343             raise CalDavSyncError(f"Не удалось получить список календарей:
344 {exc}. {_auth_scheme_hint(self.account.url)}") from exc
345         except Exception as exc:
346             raise CalDavSyncError(f"Не удалось получить список календарей: {exc}") from exc

```

```

347 def list_calendars_detailed(self) -> list[CalDavCalendarInfo]:
348     """Все календари, включая расшаренные нам коллегами (не только
349     собственные) — owner коллекции указывает на ЧУЖОЙ принципал, если
350     это так. См. CalDavCalendarInfo — задача "настроить получение
351     расшаренных календарей для VK Mail".

353     Жалоба после первой версии: "поиск находит основной, но не находит
354     расшаренный другого пользователя", при том что Evolution на том же
355     сервере его видит. Два отличия от простого PROPFIND Depth:1 по
356     одному calendar-home-set, которые Evolution (e-webdav-discover)
357     делает, а первая версия — нет: (1) calendar-home-set у принципала
358     может содержать НЕСКОЛЬКО href (свои календари в одном доме, чужие
359     расшаренные — в другом), caldav-библиотека берёт только первый;
360     (2) внутри дома могут лежать обычные под-коллекции (например,
361     shared/), в которых уже лежат календари — Depth:1 их не видит,
362     Evolution в такие коллекции спускается."""
363     try:
364         principal = self._client.principal()
365         home_urls = self._calendar_home_urls(principal)
366     except AuthorizationError as exc:
367         raise CalDavSyncError(f"Не удалось получить список календарей:
{exc}.{_auth_scheme_hint(self.account.url)}") from exc
368     except Exception as exc:
369         raise CalDavSyncError(f"Не удалось получить список календарей: {exc}") from exc

371     my_principal_path = _href_path(str(principal.url))
372     infos: list[CalDavCalendarInfo] = []
373     seen: set[str] = set()
374     for home_url in home_urls:
375         self._collect_calendars(home_url, my_principal_path, infos, seen, depth_left=2)
376     return infos

378 def _calendar_home_urls(self, principal) -> list[str]:
379     """Все href из calendar-home-set принципала (не только первый)."""
380     urls: list[str] = []
381     try:
382         response = _with_connection_retry(
383             self._client.propfind, str(principal.url), _propfind_body([_CALDAV_HOME_SET_PROP]), 0
384         )
385         for result in _parse_multistatus(response.tree):
386             value = result.properties.get(_CALDAV_HOME_SET_PROP)
387             hrefs = value if isinstance(value, list) else [value]
388             for href in hrefs:
389                 if isinstance(href, str) and href:
390                     urls.append(str(self._client.url.join(href)))
391     except Exception:
392         pass # ниже — обычный путь библиотеки, как раньше
393     if not urls:
394         urls.append(str(principal.calendar_home_set.url))
395     return urls

397 def _collect_calendars(
398     self, url: str, my_principal_path: str, infos: list[CalDavCalendarInfo], seen: set[str], depth_left: int
399 ) -> None:
400     try:
401         response = _with_connection_retry(
402             self._client.propfind, url, _propfind_body(_CALENDAR_DISCOVERY_PROPS), 1
403         )
404     except AuthorizationError as exc:
405         raise CalDavSyncError(f"Не удалось получить список календарей:
{exc}.{_auth_scheme_hint(self.account.url)}") from exc
406     except Exception as exc:
407         if depth_left == 2:
408             raise CalDavSyncError(f"Не удалось получить список календарей: {exc}") from exc
409         return # вложенная коллекция не отвечает — не роняем весь список
410     own_path = _href_path(url)
411     for result in _parse_multistatus(response.tree):
412         if result.status not in (200, 207):
413             continue
414         resourcetype = result.properties.get("{DAV:}resourcetype") or []
415         tags = [str(t) for t in (resourcetype if isinstance(resourcetype, list) else [resourcetype])]
416         full_url = str(self._client.url.join(result.href))
417         path = _href_path(full_url)
418         if path == own_path:
419             continue # сама необходимая коллекция
420         # Точное имя тега, не подстрока "calendar": у расшаренных
421         # коллекций в resourcetype бывает {http://calendarserver.org/ns/}shared
422         # — "calendarserver" содержит "calendar", ложное срабатывание.
423         is_calendar = _CALDAV_CALENDAR_TAG in tags
424         is_collection = any(t.endswith("}collection") for t in tags)
425         if is_calendar:
426             if path in seen:
427                 continue
428             seen.add(path)
429             owner_raw = result.properties.get("{DAV:}owner")
430             if isinstance(owner_raw, list):
431                 owner_raw = owner_raw[0] if owner_raw else None
432             owner = owner_raw if isinstance(owner_raw, str) and owner_raw else None
433             infos.append(CalDavCalendarInfo(

```

```

434         url=full_url,
435         name=result.properties.get("{DAV:}displayname") or result.href,
436         owner=owner,
437         is_shared=owner is not None and _href_path(owner) != my_principal_path,
438         read_only=not _has_write_privilege(result.properties.get("{DAV:}current-user-privilege-set")),
439     ))
440     elif is_collection and depth_left > 1:
441         self._collect_calendars(full_url, my_principal_path, infos, seen, depth_left - 1)

442
443     def fetch_events(self, start: datetime, end: datetime, my_email: str) -> list[Event]:
444         """События сервера в окне [start, end). Разбор VEVENT переиспользует
445         itip.parse_ics_events — ту же логику, что уже проверена на реальных
446         .ics-вложениях и импорте, вместо второго независимого парсера
447         одного и того же формата."""
448         calendar = self._primary_calendar()
449         try:
450             results = _with_connection_retry(calendar.date_search, start, end)
451         except Exception as exc:
452             raise CalDavSyncError(f"Не удалось получить события с сервера: {exc}") from exc

453
454         events: list[Event] = []
455         for obj in results:
456             try:
457                 raw = obj.data
458                 raw_bytes = raw.encode("utf-8") if isinstance(raw, str) else raw
459                 events.extend(itip.parse_ics_events(raw_bytes, my_email))
460             except Exception:
461                 continue # одно повреждённое/непонятное событие не должно валить всю синхронизацию
462         return events

463
464     def push_event(self, event: Event, organizer_email: str, organizer_name: str) -> None:
465         """Создаёт событие на сервере или обновляет уже существующее
466         (находит по UID — тот же UID, что уже используется локально)."""
467         calendar = self._primary_calendar()
468         ics_text = itip.build_caldav_ics(event, organizer_email, organizer_name).decode("utf-8")
469         try:
470             existing = _with_connection_retry(calendar.event_by_uid, event.uid)
471         except NotFoundError:
472             existing = None
473         except Exception as exc:
474             raise CalDavSyncError(f"Не удалось проверить событие на сервере: {exc}") from exc

475
476         try:
477             if existing is not None:
478                 existing.data = ics_text
479                 _with_connection_retry(existing.save)
480             else:
481                 _with_connection_retry(calendar.save_event, ics_text)
482         except Exception as exc:
483             raise CalDavSyncError(f"Не удалось сохранить событие на сервере: {exc}") from exc

484
485     def test_write_access(self) -> None:
486         """Настоящая проверка записи: создаёт одноразовое тестовое событие на
487         сервере и сразу удаляет его. В отличие от одной лишь проверки чтения
488         (PROPFIND/список календарей), это ловит именно ту ситуацию, из-за
489         которой возникла путаница — "проверка подключения проходит, а
490         синхронизация нет": проверка подключения раньше проверяла только
491         чтение, поэтому расхождение между чтением и записью (например,
492         избирательная фильтрация PUT на стороне сети по каким-то признакам
493         запроса) не обнаруживалось заранее, а всплывало только при реальной
494         синхронизации. Бросает CalDavSyncError с текстом самой сетевой
495         ошибки, если запись не удалась — чтение при этом может быть исправно."""
496         calendar = self._primary_calendar()
497         test_uid = f"redmail-conntest-{uuid4()}@redmail"
498         start = datetime.now(timezone.utc).replace(microsecond=0)
499         test_event = Event(
500             uid=test_uid,
501             summary="redmail: проверка подключения (можно удалить)",
502             dtstart=start,
503             dtend=start + timedelta(minutes=1),
504             organizer_email=self.account.username,
505             organizer_name=self.account.username,
506             is_organizer=True,
507         )
508         ics_text = itip.build_caldav_ics(test_event, test_event.organizer_email,
test_event.organizer_name).decode("utf-8")
509         try:
510             _with_connection_retry(calendar.save_event, ics_text)
511         except Exception as exc:
512             raise CalDavSyncError(f"Запись на сервер не удалась: {exc}") from exc
513         try:
514             existing = _with_connection_retry(calendar.event_by_uid, test_uid)
515             _with_connection_retry(existing.delete)
516         except Exception:
517             pass # уборка тестового события — не критично, если не получилось

518
519     def delete_event(self, uid: str) -> None:
520         calendar = self._primary_calendar()
521         try:

```



```

522         existing = _with_connection_retry(calendar.event_by_uid, uid)
523     except NotFoundError:
524         return # уже нет на сервере — нечего удалять, не ошибка
525     except Exception as exc:
526         raise CalDavSyncError(f"Не удалось найти событие на сервере: {exc}") from exc
527     try:
528         _with_connection_retry(existing.delete)
529     except Exception as exc:
530         raise CalDavSyncError(f"Не удалось удалить событие на сервере: {exc}") from exc

532     def close(self) -> None:
533         try:
534             self._client.close()
535         except Exception:
536             pass

==== src/redmail/archive_store.py (448 строк) ====
1  from __future__ import annotations

3  import mailbox
4  import sqlite3
5  from contextlib import closing
6  from email import message_from_bytes
7  from email.header import decode_header
8  from email.message import EmailMessage, Message
9  from email.utils import formatdate, parseaddr
10 from pathlib import Path

12 from redmail.imap_client import MessageContent, MessageSummary, extract_content, parse_importance

14 # Свой формат: один файл SQLite на архив. Не PST — по назначению аналог
15 # (папка с письмами, которую можно отключить от сервера и открыть локально),
16 # но не побайтово совместим с ним. Реальный .pst слишком сложен, чтобы
17 # писать самим (см. обсуждение — читаем чужие через rufpp, свои не пишем).
18 _FORMAT_VERSION = 1

20 _SCHEMA = """
21 CREATE TABLE IF NOT EXISTS meta (
22     key TEXT PRIMARY KEY,
23     value TEXT NOT NULL
24 );

26 CREATE TABLE IF NOT EXISTS messages (
27     id INTEGER PRIMARY KEY AUTOINCREMENT,
28     folder TEXT NOT NULL,
29     subject TEXT NOT NULL,
30     sender TEXT NOT NULL,
31     sender_email TEXT NOT NULL,
32     date TEXT NOT NULL,
33     message_id TEXT NOT NULL,
34     has_attachments INTEGER NOT NULL DEFAULT 0,
35     marker_color TEXT,
36     importance TEXT NOT NULL DEFAULT 'normal',
37     raw BLOB NOT NULL
38 );
39 """

42 class NotAnArchiveError(Exception):
43     """Файл существует, но это не наш формат архива (или он новее, чем мы умеем читать)."""

46 def _connect(path: Path) -> sqlite3.Connection:
47     conn = sqlite3.connect(path)
48     conn.executescript(_SCHEMA)
49     return conn

52 def create_archive(path: Path) -> None:
53     """Создаёт пустой файл архива. Не трогает уже существующий по этому пути."""
54     path.parent.mkdir(parents=True, exist_ok=True)
55     with closing(_connect(path)) as conn:
56         row = conn.execute("SELECT value FROM meta WHERE key = 'format_version'").fetchone()
57         if row is None:
58             conn.execute(
59                 "INSERT INTO meta (key, value) VALUES ('format_version', ?)", (str(_FORMAT_VERSION),)
60             )
61         conn.commit()

64 def is_archive_file(path: Path) -> bool:
65     """Дешёвая проверка перед тем, как предлагать открыть файл как архив."""
66     if not path.exists():
67         return False
68     try:
69         with closing(sqlite3.connect(path)) as conn:
70             row = conn.execute("SELECT value FROM meta WHERE key = 'format_version'").fetchone()
71     except sqlite3.DatabaseError:
72         return False

```

```

73     return row is not None

76 def list_folders(path: Path) -> list[str]:
77     with closing(_connect(path)) as conn:
78         rows = conn.execute("SELECT DISTINCT folder FROM messages ORDER BY folder").fetchall()
79         return [row[0] for row in rows]

82 def rename_folder(path: Path, old_name: str, new_name: str) -> None:
83     # У архива нет отдельной таблицы папок — "папка" это просто текстовое
84     # поле у писем (см. схему выше), так что переименование — это
85     # переименование значения этого поля у всех писем сразу. Если
86     # new_name совпадает с уже существующей папкой, письма просто
87     # сольются в неё — это ожидаемо, отдельно не запрещаем.
88     #
89     # Дерево в интерфейсе строится из этих строк разбиением по "/" — у
90     # промежуточного узла ("Работа" для писем в "Работа/Проекты") нет
91     # отдельной записи, только у его вложенных подпапок. Чтобы
92     # переименование такого узла вообще что-то делало, переименовываем и
93     # все вложенные "old_name/..." тем же префиксом.
94     with closing(_connect(path)) as conn:
95         conn.execute("UPDATE messages SET folder = ? WHERE folder = ?", (new_name, old_name))
96         conn.execute(
97             "UPDATE messages SET folder = ? || substr(folder, ?) WHERE folder LIKE ? ESCAPE '\\'",
98             (new_name, len(old_name) + 1, old_name.replace("\\", "\\\\" ).replace("%", "\\%").replace("_", "\\_") +
99             "/%"),
100             conn.commit()

103 def list_messages(path: Path, folder: str) -> list[MessageSummary]:
104     with closing(_connect(path)) as conn:
105         rows = conn.execute(
106             "SELECT id, subject, sender, sender_email, date, message_id, has_attachments, marker_color, importance
107             FROM messages WHERE folder = ? ORDER BY date DESC, id DESC",
108             (folder,)),
109         ).fetchall()
110     return [
111         MessageSummary(
112             uid=row[0], subject=row[1], sender=row[2], sender_email=row[3], date=row[4], message_id=row[5],
113             has_attachments=bool(row[6]), marker_color=row[7], importance=row[8],
114             is_read=True, # архив — уже разобранный почта, не показываем "непрочитанным" вечно
115         )
116         for row in rows
117     ]

120 def get_message_content(path: Path, message_id: int) -> MessageContent:
121     return extract_content(message_from_bytes(get_message_raw(path, message_id)))

124 def get_message_raw(path: Path, message_id: int) -> bytes:
125     with closing(_connect(path)) as conn:
126         row = conn.execute("SELECT raw FROM messages WHERE id = ?", (message_id,)).fetchone()
127         if row is None:
128             raise KeyError(f"В архиве нет письма с id={message_id}")
129         return row[0]

132 def set_marker(path: Path, message_id: int, color: str | None) -> None:
133     with closing(_connect(path)) as conn:
134         conn.execute("UPDATE messages SET marker_color = ? WHERE id = ?", (color, message_id))
135         conn.commit()

138 def delete_messages(path: Path, message_ids: list[int]) -> None:
139     if not message_ids:
140         return
141     placeholders = ",".join("?" * len(message_ids))
142     with closing(_connect(path)) as conn:
143         conn.execute(f"DELETE FROM messages WHERE id IN ({placeholders})", message_ids)
144         conn.commit()

147 def append_raw_message(path: Path, folder: str, raw: bytes) -> int:
148     """Разбирает raw (полное письмо в формате RFC 822) и добавляет в архив."""
149     with closing(_connect(path)) as conn:
150         row_id = _insert_raw(conn, folder, raw)
151         conn.commit()
152     return row_id

155 def _insert_raw(conn: sqlite3.Connection, folder: str, raw: bytes) -> int:
156     message = message_from_bytes(raw)
157     sender_display, sender_email = _decode_from(message)
158     content = extract_content(message)
159     cursor = conn.execute(

```

```

160         "INSERT INTO messages (folder, subject, sender, sender_email, date, message_id, "
161         "has_attachments, marker_color, importance, raw) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
162         (
163             folder,
164             _decode_mime_words(message.get("Subject")) or "(без темы)",
165             sender_display,
166             sender_email,
167             _format_date(message.get("Date")),
168             message.get("Message-Id", "") or "",
169             int(bool(content.attachments)),
170             None,
171             parse_importance(message),
172             raw,
173         ),
174     )
175     return cursor.lastrowid

178 def _decode_mime_words(value: str | None) -> str:
179     if not value:
180         return ""
181     try:
182         parts = decode_header(value)
183     except Exception:
184         # decode_header() поднимает HeaderParseError на действительно
185         # повреждённом base64/QP (не только на пустой строке) – реальный
186         # случай: PST-свойство "имя отправителя" иногда хранит RFC2047-имя
187         # ОБРЕЗАННЫМ (см. _pst_message_to_raw), и обрезка иногда режет
188         # прямо посреди base64-группы, оставляя невалидные символы. Без
189         # этого один такой отправитель падал бы всем импортом .pst разом
190         # (жалоба: "загрузка из pst все ещё некорректно загружает...
191         # авторов") – лучше показать значение как есть, чем потерять
192         # письмо или весь импорт целиком.
193     return value
194     return "".join(
195         chunk.decode(encoding or "utf-8", errors="replace") if isinstance(chunk, bytes) else chunk
196         for chunk, encoding in parts
197     )

200 def _decode_from(message: Message) -> tuple[str, str]:
201     name, addr = parseaddr(message.get("From", ""))
202     decoded_name = _decode_mime_words(name)
203     if decoded_name:
204         return decoded_name, addr
205     # Если в "From" нет реального адреса в <угловых скобках> – только
206     # закодированное отображаемое имя целиком – parseaddr() кладёт всю
207     # строку в "адрес" (RFC 822 не делит её на имя+адрес без <>), и она
208     # так и остаётся закодированной как =?utf-8?b?...?=?, потому что
209     # decode_header() выше применялся только к "имени". Найдено на
210     # реальном PST (жалоба: "не распознались адреса отправителей в архиве").
211     decoded_addr = _decode_mime_words(addr)
212     if decoded_addr != addr:
213         # "Адрес" сам оказался закодированным именем, не настоящим
214         # e-mail – реального адреса в письме не было.
215         return decoded_addr or "(неизвестно)", ""
216     return addr or "(неизвестно)", addr

219 def _format_date(raw_date: str | None) -> str:
220     if not raw_date:
221         return ""
222     from email.utils import parsedate_to_datetime
223
224     try:
225         parsed = parsedate_to_datetime(raw_date)
226     except (TypeError, ValueError):
227         return ""
228     if parsed is None:
229         return ""
230     return parsed.strftime("%Y-%m-%d %H:%M")

233 # -----
234 # Импорт из внешних форматов
235 # -----

238 def import_mbox(path: Path, mbox_file: Path, folder: str) -> int:
239     """Импортирует все письма из mbox-файла (Evolution, Thunderbird и т.п.)."""
240     create_archive(path)
241     count = 0
242     box = mailbox.mbox(str(mbox_file))
243     try:
244         with closing(_connect(path)) as conn:
245             for message in box:
246                 _insert_raw(conn, folder, message.as_bytes())
247                 count += 1
248     except:
249         conn.commit()

```

```

249         finally:
250             box.close()
251         return count

254 def import_maildir(path: Path, maildir_dir: Path, folder: str) -> int:
255     """Импортирует Maildir-каталог (формат хранения почты Evolution на диске)."""
256     create_archive(path)
257     count = 0
258     box = mailbox.Maildir(str(maildir_dir), create=False)
259     try:
260         with closing(_connect(path)) as conn:
261             for message in box:
262                 _insert_raw(conn, folder, message.as_bytes())
263                 count += 1
264             conn.commit()
265     finally:
266         box.close()
267     return count

270 _ATTACHMENT_FILENAME_TAGS = (0x3707, 0x3704) # PidTagAttachLongFilename, PidTagAttachFilename

272 # Служебные уровни, которые MAPI/PST всегда создаёт над реальными папками
273 # пользователя (корень хранилища — имя пустое, а под ним ровно одна папка
274 # с одним из этих стандартных имён) — Outlook сам их никогда не показывает,
275 # сразу открывая содержимое "Top of Personal Folders" как верхний уровень.
276 # Раньше архив показывал оба уровня как настоящие папки (жалоба
277 # пользователя: "зачем эта структура?", видел в дереве
278 # "(без имени)/Top of Personal Folders/Входящие" вместо просто "Входящие").
279 _PST_WRAPPER_FOLDER_NAMES = {
280     "top of personal folders",
281     "top of information store",
282     "root - mailbox",
283     "root - mailbox data",
284 }

286 # Папки, чьё содержимое — не почта (события календаря, карточки контактов и
287 # т.п.), а другие типы MAPI-элементов. get_plain_text_body()/get_subject() и
288 # т.д. на таких элементах либо пусты, либо не имеют смысла как "письмо" —
289 # раньше они всё равно попадали в архив как обычные (пустые/бессмысленные)
290 # письма (жалоба: "календарь загрузился как письма, контакты тоже как
291 # письма"). rpyff не даёт типизированного доступа к событиям/контактам
292 # (только к обычным message-элементам с почтовыми полями), поэтому такие
293 # папки при импорте пропускаются целиком, а не превращаются в мусорные
294 # "письма" — полноценный перенос PST-календаря/контактов в модули
295 # Календарь/Контакты это отдельная, более крупная задача.
296 _PST_NON_MAIL_FOLDER_NAMES = {
297     "календарь", "calendar",
298     "контакты", "contacts",
299     "задачи", "tasks",
300     "заметки", "notes", "sticky notes",
301     "журнал", "journal",
302 }

305 def import_pst(path: Path, pst_file: Path) -> int:
306     """Импортирует .pst (Outlook), сохраняя структуру папок исходного файла.

308     PST хранит письма как MAPI-свойства, не как готовый RFC 822 — для
309     писем, полученных по почте, часто есть исходные "сырые" заголовки
310     (transport headers), но не для писем, составленных прямо в Outlook.
311     Поэтому raw для архива собираем сами через email.message.EmailMessage
312     из текста/отправителя/темы/вложений — так работает единообразно для
313     обоих случаев, а остальной код архива (чтение, отображение) не должен
314     знать, что письмо изначально не было "настоящим" RFC 822.
315     """
316     try:
317         import pypff
318     except ImportError as exc:
319         raise RuntimeError(
320             "Для импорта .pst нужен пакет libpff-python (модуль pypff) — "
321             "он не установлен в этом окружении."
322         ) from exc

324     create_archive(path)
325     count = 0
326     pst = pypff.file()
327     pst.open(str(pst_file))
328     try:
329         with closing(_connect(path)) as conn:
330             count = _import_pst_folder(conn, pst.get_root_folder(), "")
331             conn.commit()
332     finally:
333         pst.close()
334     return count

337 def _import_pst_folder(conn: sqlite3.Connection, folder, path_prefix: str) -> int:

```

```

338     count = 0
339     raw_name = _safe_pst_text(folder.get_name()).strip()
340     is_wrapper = not path_prefix and (not raw_name or raw_name.lower() in _PST_WRAPPER_FOLDER_NAMES)
341     is_non_mail = raw_name.lower() in _PST_NON_MAIL_FOLDER_NAMES

342     if is_wrapper:
343         # Служебный уровень хранилища – не настоящая папка пользователя,
344         # пропускаем его в пути, но всё равно спускаемся в подпапки (там и
345         # лежат реальные "Входящие"/"Отправленные"/...). "not path_prefix"
346         # защищает от совпадения с реальной пользовательской подпапкой
347         # где-то глубже – эти служебные имена MAPI создаёт только на самом
348         # верхнем уровне хранилища.
349         current_path = path_prefix
350     else:
351         folder_name = raw_name or "(без имени)"
352         current_path = f"{path_prefix}/{folder_name}" if path_prefix else folder_name

353     if not is_wrapper and not is_non_mail:
354         for message in folder.sub_messages:
355             try:
356                 raw = _pst_message_to_raw(message)
357                 _insert_raw(conn, current_path or "Импорт из PST", raw)
358             except Exception:
359                 # Одно повреждённое/нестандартное письмо не должно ронять весь
360                 # импорт (жалоба: "загрузка из pst все ещё некорректно
361                 # загружает имена папок и авторов" – раньше на таком письме
362                 # мог упасть весь импорт разом, теряя и все остальные,
363                 # корректные письма вместе с ним).
364                 continue
365             count += 1
366     for sub_folder in folder.sub_folders:
367         count += _import_pst_folder(conn, sub_folder, current_path)
368     return count

369 def _safe_pst_text(value) -> str:
370     """pyppff обычно отдаёт уже готовую Python-строку, но на нестандартных
371     .pst встречались случаи, когда свойство отдаёт bytes или бросает
372     исключение при доступе – не даём этому уронить весь импорт."""
373     if value is None:
374         return ""
375     if isinstance(value, bytes):
376         return value.decode("utf-8", errors="replace")
377     return str(value)

378 def _sanitize_header_value(value: str) -> str:
379     """message_from_bytes() без явной политики использует compat32, который
380     отдаёт значения заголовков как есть – включая необёрнутые переводы
381     строк из многострочных (folded) заголовков. Строгая политика
382     EmailMessage по умолчанию такое не принимает ("Header values may not
383     contain linefeed..."). Найдено на реальном PST-файле – часть писем в
384     нём падала именно на этом при импорте To/Cc/Message-Id."""
385     return " ".join(str(value).split())

386 def _pst_message_to_raw(message) -> bytes:
387     email_message = EmailMessage()
388     email_message["Subject"] = _sanitize_header_value(message.get_subject() or "")
389     email_message["From"] = _sanitize_header_value(message.get_sender_name() or "")

390     delivery_time = message.get_delivery_time()
391     if delivery_time is not None:
392         email_message["Date"] = formatdate(delivery_time.timestamp())

393     transport_headers = message.get_transport_headers()
394     if transport_headers:
395         # Письмо реально пришло по почте – там уже есть настоящие заголовки.
396         # Найдено на реальном PST: MAPI-свойство "имя отправителя"
397         # (get_sender_name) иногда хранит RFC2047-имя ОБРЕЗАННЫМ (похоже,
398         # PST/MAPI не сворачивает несколько encoded-word кусков в одно
399         # свойство так же, как это делает настоящий заголовок письма) – в
400         # то время как то же имя в самом заголовке From: из
401         # transport_headers сворачивается (folding) корректно и
402         # разбирается полностью. Поэтому From (и Subject туда же, для
403         # единообразия) берём из настоящих заголовков, когда они есть.
404         parsed_headers = message_from_bytes(transport_headers.encode("utf-8", errors="replace"))
405         for header in ("Message-Id", "To", "Cc", "From", "Subject"):
406             value = parsed_headers.get(header)
407             if value:
408                 del email_message[header]
409                 email_message[header] = _sanitize_header_value(value)

410     body = message.get_plain_text_body()
411     if not body:
412         html_body = message.get_html_body()
413         body = "(письмо в формате HTML – предпросмотр текста недоступен)" if html_body else "(нет текстового
содержимого)"
414     elif isinstance(body, bytes):

```

```

426         body = body.decode("utf-8", errors="replace")
427         email_message.set_content(body)

429         attachment_count = message.get_number_of_attachments()
430         for index in range(attachment_count):
431             attachment = message.get_attachment(index)
432             data = attachment.read_buffer(attachment.get_size())
433             filename = _pst_attachment_filename(attachment, index)
434             email_message.add_attachment(data, maintype="application", subtype="octet-stream", filename=filename)

436         return bytes(email_message)

439 def _pst_attachment_filename(attachment, index: int) -> str:
440     for i in range(attachment.number_of_record_sets):
441         record_set = attachment.get_record_set(i)
442         for j in range(record_set.number_of_entries):
443             entry = record_set.get_entry(j)
444             if entry.get_entry_type() in _ATTACHMENT_FILENAME_TAGS:
445                 name = entry.get_data_as_string()
446                 if name:
447                     return name
448         return f"attachment-{index + 1}"

==== src/redmail/calendar_store.py (503 строк, показаны первые 25) ====
1  from __future__ import annotations

3  import json
4  import sqlite3
5  from contextlib import closing
6  from dataclasses import dataclass, field, replace
7  from datetime import datetime
8  from pathlib import Path
9  from uuid import uuid4

11 from dateutil.rrule import rrulestr

13 from redmail.imap_client import Attachment

15 # Свой формат: один файл SQLite на календарь — та же идея, что у архива
16 # писем (archive_store.py). Сервер календаря (Exchange/VK Mail) не нужен:
17 # в закрытой корпоративной среде без CalDAV/EWS-доступа события хранятся
18 # локально, а встречи/ответы/переносы передаются через обычную почту как
19 # .ics-вложения (iTIP, RFC 5546) — см. itip.py.
20 #
21 # В отличие от почтового кэша (cache_store.py), при смене схемы этот файл
22 # НЕЛЬЗЯ просто стирать и пересобирать — это единственная копия локально
23 # принятых приглашений и созданных пользователем встреч, а не что-то
24 # регенерируемое с сервера. Поэтому новые поля добавляются через
25 # ALTER TABLE-миграции (см. _MIGRATIONS), а не через wipe-on-mismatch.

```